



UNIVERSIDADE FEDERAL DO AMAPÁ
DEPARTAMENTO DE CIÊNCIAS EXATAS E TECNOLÓGICAS
CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

VITOR HUGO AMANAJÁS ROCHA
FELIPE DA SILVA RODRIGUES

PROPOSTA DE UM MANUAL DE IMPLEMENTAÇÃO DA CLEAN
ARCHITECTURE EM PROJETOS NODE.JS COM TYPESCRIPT

MACAPÁ – AP

2026

VITOR HUGO AMANAJÁS ROCHA

FELIPE DA SILVA RODRIGUES

**PROPOSTA DE UM MANUAL DE IMPLEMENTAÇÃO DA CLEAN
ARCHITECTURE EM PROJETOS NODE.JS COM TYPESCRIPT**

Trabalho de Conclusão de Curso submetido à Banca
Examinadora do Curso de Ciência da Computação da
Universidade Federal do Amapá para a obtenção do
título de Bacharel em Ciência da Computação.

Orientador: Prof. Dr. Julio Cezar Costa Furtado.

MACAPÁ – AP

2026

Dados Internacionais de Catalogação na Publicação (CIP)
Biblioteca Central/UNIFAP-Macapá-AP
Elaborado por Cristina Fernandes – CRB-2 / 1569

R672p Rocha, Vitor Hugo Amanajás.

Proposta de um manual de implementação da clean architecture em projetos node.js com typescript. / Vitor Hugo Amanajás Rocha, Felipe da Silva Rodrigues. - Macapá, 2026.
1 recurso eletrônico.
85 f.

Trabalho de Conclusão de Curso (Graduação em Ciências da Computação), Universidade Federal do Amapá, Macapá, 2026.

Orientador: Julio Cezar Costa Furtado.

Modo de acesso: World Wide Web.

Formato de arquivo: Portable Document Format (PDF).

1. Clean Architecture. 2. Arquitetura de software. 3. Node.js. I. Furtado, Julio Cezar Costa, orientador. II. Universidade Federal do Amapá. III. Título.

CDD 23. ed. – 005.1

ROCHA, Vitor Hugo Amanajás; RODRIGUES, Felipe da Silva. **Proposta de um manual de implementação da clean architecture em projetos node.js com typescript**. Orientador: Julio Cezar Costa Furtado. 2026. 85 f. Trabalho de Conclusão de Curso (Graduação) - Ciências da Computação, Macapá, 2026.



UNIVERSIDADE FEDERAL DO AMAPÁ
DEPARTAMENTO DE CIÊNCIAS EXATAS E TECNOLÓGICAS
COORDENAÇÃO DO CURSO DE CIÊNCIA DA COMPUTAÇÃO

ATA DE DEFESA DE TCC

Realizou-se no dia 24 de julho de 2026, às 19h00, por videoconferência, a defesa do TCC intitulado: “PROPOSTA DE UM MANUAL DE IMPLEMENTAÇÃO DA CLEAN ARCHITECTURE EM PROJETOS NODE.JS COM TYPESCRIPT”, dos discentes Vitor Hugo Amanajás Rocha, matrícula 2018009814, e Felipe da Silva Rodrigues, matrícula 2018026904. A Banca Examinadora foi composta pelo Prof. Dr. JULIO CEZAR COSTA FURTADO, presidente da banca e orientador; Prof. MSc. MARCO ANTÔNIO LEAL DA SILVA e Prof. MSc. THIAGO PINHEIRO DO NASCIMENTO, examinadores. Concluída a defesa, foram realizadas as arguições e comentários. Em seguida, procedeu-se o julgamento pelos membros da Banca Examinadora, tendo o trabalho sido APROVADO com nota 10,0 (dez).

E, para constar, eu, Prof. Dr. JULIO CEZAR COSTA FURTADO, orientador e presidente da Banca Examinadora, lavrei a presente ata que, após lida e achada conforme, foi assinada por mim e demais membros da Banca Examinadora.

Macapá, 24 de julho de 2026.

Documento assinado digitalmente
gov.br JULIO CEZAR COSTA FURTADO
Data: 10/08/2026 13:18:43-0300
Verifique em <https://validar.iti.gov.br>

Prof. Dr. JULIO CEZAR COSTA FURTADO
Orientador do TCC

Documento assinado digitalmente
gov.br MARCO ANTONIO LEAL DA SILVA
Data: 12/08/2026 13:57:41-0300
Verifique em <https://validar.iti.gov.br>

Prof. MSc. MARCO ANTÔNIO LEAL DA SILVA
Examinador (UNIFAP)

Documento assinado digitalmente
gov.br THIAGO PINHEIRO DO NASCIMENTO
Data: 09/08/2026 12:58:08-0300
Verifique em <https://validar.iti.gov.br>

Prof. MSc. THIAGO PINHEIRO DO
NASCIMENTO
Examinador (UNIFAP)

Por Cristo, com Cristo e em Cristo.

AGRADECIMENTOS

Vitor - Agradeço primeiramente a Deus, pelas bênçãos e desafios colocados em minha vida e pelas pessoas que colocou em meu caminho para que eu pudesse chegar até aqui.

Agradeço aos meus amados pais, Ivonaldo e Priscila, que nunca mediram esforços para me proporcionar tempo, apoio e acesso à educação, contribuindo de forma essencial para minha formação pessoal e acadêmica.

Agradeço à minha querida esposa, Monique, pelo apoio, pela compreensão e por estar ao meu lado durante esta trajetória.

Agradeço também ao meu professor e orientador, Prof. Dr. Júlio Cezar Costa Furtado, pela paciência, pelos ensinamentos e pela orientação dedicada durante o desenvolvimento deste trabalho.

Felipe - Agradeço a Deus, por me conceder a vida, saúde e discernimento, e por me fazer perseverar diante das adversidades ao longo desta jornada.

Agradeço aos meus pais, Fabiane e Michel, pelo amor, apoio e dedicação, por nunca deixarem faltar o necessário e por me criarem com valores que contribuíram para que eu me tornasse quem sou hoje.

Agradeço ao meu orientador, Prof. Dr. Júlio Cezar Costa Furtado, pela paciência e pelos ensinamentos que contribuíram para o meu desenvolvimento acadêmico e profissional.

Agradeço também aos amigos e colegas que, de alguma forma, fizeram parte desta caminhada, pelo apoio, incentivo e companheirismo ao longo desta trajetória.

RESUMO

A crescente complexidade dos sistemas de software exige abordagens arquiteturais que promovam manutenibilidade, testabilidade e independência de tecnologias específicas. Nesse contexto, a Clean Architecture, proposta por Robert C. Martin, apresenta um modelo que organiza o software em camadas bem definidas, com regras claras de dependência, visando proteger o núcleo de regras de negócio contra mudanças externas. Este trabalho tem como objetivo propor um manual de implementação da Clean Architecture em projetos Node.js com TypeScript, demonstrado por meio do desenvolvimento de uma API RESTful. Para atingir esse objetivo, foi desenvolvido um sistema de gerenciamento de listas de tarefas, no qual usuários podem criar listas e gerenciar tarefas associadas. O projeto foi estruturado de acordo com os princípios da Clean Architecture, separando as camadas de entidades, casos de uso, adaptadores de interface e infraestrutura, e cada camada foi implementada de modo a respeitar a regra de dependência, mantendo o domínio e a lógica da aplicação independentes de frameworks, mecanismos de persistência e protocolos de comunicação. A avaliação, de natureza qualitativa e baseada em um estudo de caso único, apoiou-se em evidências extraídas do próprio código e em indicadores estruturais. Os resultados observados apontam maior clareza estrutural e indícios de menor acoplamento entre componentes, além de um núcleo passível de verificação isolada, sem dependência de banco de dados ou de framework, o que favorece a testabilidade. A avaliação indica que a Clean Architecture é aplicável mesmo em sistemas de menor complexidade, ainda que ao custo de maior esforço inicial de organização. A sistematização das decisões de implementação, das boas práticas e dos erros frequentes em formato de manual oferece um material de referência para estudantes e desenvolvedores. Assim, o trabalho discute, na prática, tanto os benefícios quanto os limites da aplicação de princípios arquiteturais bem definidos no desenvolvimento de sistemas backend modernos.

Palavras-chave: Clean Architecture. Arquitetura de software. Node.js. TypeScript. API RESTful.

ABSTRACT

The increasing complexity of software systems requires architectural approaches that improve maintainability, testability and independence from specific technologies. In this context, Clean Architecture, proposed by Robert C. Martin, organizes software into well-defined layers and establishes dependency rules to protect business rules from external changes. This work aims to propose an implementation manual for Clean Architecture in Node.js projects with TypeScript, demonstrated through the development of a RESTful API. To achieve this objective, a task list management system was developed. In this system, users can create lists and manage related tasks. The project was structured according to the principles of Clean Architecture, separating entities, use cases, interface adapters and infrastructure, and each layer was implemented to respect the dependency rule, keeping the domain and application logic independent from frameworks, persistence mechanisms and communication protocols. The evaluation, qualitative in nature and based on a single case study, relied on evidence drawn from the source code and on structural indicators. The observed results point to clearer structure and signs of lower coupling between components, as well as a core that can be verified in isolation, without depending on a database or framework, which favors testability. The analysis indicates that Clean Architecture can be applied even in systems with simple business rules, although at the cost of greater initial organization effort. Systematizing the implementation decisions, good practices and common mistakes in the form of a manual provides reference material for students and developers. Thus, the work discusses, in practice, both the benefits and the limits of applying well-defined architectural principles in the development of modern backend software.

Keywords: Clean Architecture. Software Architecture. Node.js. TypeScript. RESTful API.

LISTA DE FIGURAS

Figura 1 - Etapas metodológicas da pesquisa.....	19
Figura 2 - Dependência direta e inversão de dependência.....	24
Figura 3 - Diagrama de camadas da Clean Architecture.....	25
Figura 4 - Fluxo de uma requisição através das camadas da Clean Architecture.....	28
Figura 5 - Modelo de domínio do sistema.....	34
Figura 6 - Endpoints da API e casos de uso correspondentes.....	37
Figura 7 - Estrutura de diretórios do projeto.....	39
Figura 8 - Interface TaskListProps.....	42
Figura 9 - Definição da classe TaskList e atributos internos.....	43
Figura 10 - Construtor da entidade TaskList.....	43
Figura 11 - Método de fábrica create da entidade TaskList.....	44
Figura 12 - Interface TaskProps.....	44
Figura 13 - Definição da classe Task e atributos internos.....	44
Figura 14 - Construtor da entidade Task.....	45
Figura 15 - Método de fábrica create da entidade Task.....	45
Figura 16 - Método toggleCompleted.....	45
Figura 17 - Validação da descrição da tarefa.....	46
Figura 18 - Definição do objeto de valor UuidVO.....	47
Figura 19 - Métodos de criação do UuidVO.....	47
Figura 20 - Validação do identificador.....	47
Figura 21 - Comparação e exposição controlada do valor.....	48
Figura 22 - Definição do caso de uso CreateTaskUseCase.....	51
Figura 23 - Validação da existência da lista de tarefas.....	51
Figura 24 - Criação da entidade Task.....	51
Figura 25 - Persistência e retorno do caso de uso CreateTask.....	52
Figura 26 - Definição do caso de uso ToggleTaskCompletionUseCase.....	52
Figura 27 - Recuperação da entidade Task.....	53
Figura 28 - Alteração de estado e persistência da tarefa.....	53
Figura 29 - Definição do caso de uso GetTasksByListUseCase.....	54
Figura 30 - Execução da listagem de tarefas.....	54
Figura 31 - Interface TaskRepository.....	56
Figura 32 - Interface TaskListRepository.....	57

Figura 33 - Tokens de repositório.....	57
Figura 34 - Uso da interface TaskRepository em um caso de uso.....	58
Figura 35 - Definição do controller de tarefas.....	58
Figura 36 - Método de criação de tarefa no controller.....	59
Figura 37 - Schema de validação para criação de tarefa.....	59
Figura 38 - Pipe de validação com Zod.....	60
Figura 39 - Guard de identificação do usuário.....	60
Figura 40 - Validação do x-user-id.....	61
Figura 41 - Filtro centralizado de exceções.....	62
Figura 42 - Conversão de exceções em respostas HTTP.....	63
Figura 43 - Serviço de integração com Prisma.....	65
Figura 44 - Repositório concreto de tarefas com Prisma.....	65
Figura 45 - Persistência da entidade Task.....	66
Figura 46 - Recuperação e restauração da entidade Task.....	67
Figura 47 - Repositório concreto de listas de tarefas.....	67
Figura 48 - Persistência da entidade TaskList.....	68
Figura 49 - Listagem de listas por usuário.....	68
Figura 50 - Módulo de repositórios.....	69
Figura 51 - Módulo de casos de uso.....	70
Figura 52 - Objeto de transferência de dados de uma tarefa.....	72
Figura 53 - Mapeador da entidade Task para objeto de transferência.....	73
Figura 54 - Repositório de tarefas em memória para testes.....	74
Figura 55 - Teste de unidade do caso de uso de alternância de conclusão.....	74

LISTA DE QUADROS

Quadro 1 - Síntese da concretização dos princípios da Clean Architecture.....	80
---	----

SUMÁRIO

1 INTRODUÇÃO.....	14
1.1 CONTEXTUALIZAÇÃO.....	14
1.2 PROBLEMA.....	15
1.3 JUSTIFICATIVA.....	16
1.4 MOTIVAÇÃO.....	17
1.5 OBJETIVOS.....	17
1.5.1 Objetivo Geral.....	17
1.5.2 Objetivos Específicos.....	18
1.6 METODOLOGIA.....	18
1.7 ESTRUTURA DO TRABALHO.....	20
2 FUNDAMENTAÇÃO TEÓRICA.....	21
2.1 ARQUITETURA DE SOFTWARE.....	21
2.2 PROBLEMAS DE ACOPLAMENTO.....	22
2.3 CLEAN ARCHITECTURE.....	24
2.4 CONCEITOS-CHAVE DA CLEAN ARCHITECTURE.....	26
2.4.1 Entidades.....	26
2.4.2 Casos de Uso.....	26
2.4.3 Interfaces.....	27
2.4.4 Adaptadores.....	27
2.5 PRINCÍPIOS E PADRÕES DE APOIO.....	28
2.5.1 Princípios SOLID.....	28
2.5.2 Padrões de persistência e de transferência de dados.....	29
2.6 TRABALHOS RELACIONADOS.....	29
2.7 SÍNTESE DOS CONCEITOS.....	31
3 APLICAÇÃO PRÁTICA E TECNOLOGIAS ADOTADAS.....	33
3.1 DESCRIÇÃO DO SISTEMA PROPOSTO.....	33
3.2 FUNCIONALIDADES DO SISTEMA.....	34
3.3 CONTEXTO DE USUÁRIO E IDENTIFICAÇÃO POR X-USER-ID.....	35
3.4 ARQUITETURA DA API.....	36
3.5 TECNOLOGIAS E FERRAMENTAS UTILIZADAS.....	38
4 MANUAL DE IMPLEMENTAÇÃO DA CLEAN ARCHITECTURE.....	41
4.1 ENTIDADES.....	41

4.1.1 Entidade TaskList.....	42
4.1.2 Entidade Task.....	44
4.1.3 Objeto de Valor UuidVO e identidade no domínio.....	46
4.1.4 Síntese da camada de entidades.....	48
4.2 CASOS DE USO.....	49
4.2.1 Estrutura geral dos casos de uso no projeto.....	50
4.2.2 Caso de uso representativo I: CreateTask.....	50
4.2.3 Caso de uso representativo II: ToggleTaskCompletion.....	52
4.2.4 Caso de uso representativo III: GetTasksByList.....	53
4.2.5 Síntese da camada de casos de uso.....	54
4.3 ADAPTADORES DE INTERFACE.....	55
4.3.1 Interfaces e portas de saída.....	56
4.3.2 Tokens de injeção de dependência.....	57
4.3.3 Uso das portas nos casos de uso.....	57
4.3.4 Controllers como adaptadores de entrada.....	58
4.3.5 Validação de dados na borda com Pipes.....	59
4.3.6 Identificação do usuário via Guard (x-user-id).....	60
4.3.7 Tratamento centralizado de exceções.....	61
4.3.8 Síntese da camada de adaptadores de interface.....	63
4.4 INFRAESTRUTURA.....	64
4.4.1 PrismaService e acesso ao banco de dados.....	64
4.4.2 Repositório concreto de tarefas.....	65
4.4.3 Repositório concreto de listas de tarefas.....	67
4.4.4 Módulo de repositórios e composição dos contratos.....	68
4.4.5 Módulo de casos de uso e disponibilização da aplicação.....	69
4.4.6 Síntese da camada de infraestrutura.....	70
4.5 TRAVESSIA DE FRONTEIRAS COM OBJETOS DE TRANSFERÊNCIA E MAPEADORES.....	71
4.6 TESTABILIDADE NA PRÁTICA.....	73
5 RESULTADOS E ANÁLISE.....	76
5.1 INDEPENDÊNCIA E SUBSTITUIBILIDADE DO NÚCLEO.....	76
5.2 MANUTENIBILIDADE E CLAREZA ESTRUTURAL.....	77
5.3 TESTABILIDADE.....	78
5.4 ISOLAMENTO DAS PREOCUPAÇÕES DE BORDA.....	78

5.5 ATENDIMENTO AOS OBJETIVOS DA PESQUISA.....	79
5.6 LIMITAÇÕES E AMEAÇAS À VALIDADE.....	80
5.7 SÍNTESE DOS RESULTADOS.....	80
6 CONCLUSÃO.....	82
REFERÊNCIAS.....	84

1 INTRODUÇÃO

O desenvolvimento de sistemas de software tem se tornado cada vez mais estratégico em diferentes áreas da sociedade, impulsionado pela crescente demanda por aplicações digitais e pela rápida evolução tecnológica. Nesse contexto, a qualidade do software passa a depender não apenas da implementação de funcionalidades, mas também da forma como o sistema é estruturado. A organização adequada do código e a definição de uma arquitetura consistente tornam-se fatores fundamentais para garantir a sustentabilidade, a manutenibilidade e a evolução das aplicações ao longo do tempo. Este capítulo apresenta o contexto em que o trabalho se insere, o problema investigado, a justificativa e a motivação do estudo, os objetivos perseguidos, a metodologia adotada e a organização do restante do documento.

1.1 CONTEXTUALIZAÇÃO

O desenvolvimento de software moderno enfrenta desafios crescentes relacionados à complexidade dos sistemas, à evolução contínua de requisitos e à rápida obsolescência das tecnologias utilizadas. Em aplicações backend, especialmente aquelas construídas com frameworks amplamente difundidos, é comum observar a presença de acoplamento excessivo entre regras de negócio e detalhes de infraestrutura, como mecanismos de persistência, frameworks e protocolos de comunicação. Esse cenário compromete atributos de qualidade essenciais, como manutenibilidade, testabilidade e capacidade de evolução do sistema ao longo do tempo (PRESSMAN; MAXIM, 2021).

Parte dessas dificuldades é capturada pela metáfora da dívida técnica, que descreve o efeito acumulado de decisões de implementação tomadas para acelerar entregas no curto prazo em detrimento da qualidade estrutural do código (CUNNINGHAM, 1992). À medida que essas decisões se acumulam sem o devido tratamento, o custo de modificação do sistema tende a crescer, comprometendo a capacidade de resposta das equipes a novas demandas (KRUCHTEN; NORD; OZKAYA, 2012). Nesse sentido, decisões estruturais tomadas nas fases iniciais de um projeto exercem influência duradoura sobre o custo de manutenção e de evolução do software.

Diante desse contexto, a arquitetura de software assume um papel central na organização dos sistemas, ao estabelecer diretrizes para a separação de responsabilidades e o controle das dependências entre componentes. Segundo Bass, Clements e Kazman (2021), a arquitetura de

software influencia diretamente atributos de qualidade e, consequentemente, a sustentabilidade e a evolução das aplicações.

A literatura também registra que, mesmo quando uma arquitetura é definida no início do projeto, a estrutura efetivamente implementada tende a se afastar progressivamente da estrutura planejada, fenômeno conhecido como erosão ou degradação arquitetural (PERRY; WOLF, 1992). Revisões sistemáticas sobre o tema indicam que a evolução acelerada dos sistemas, a frequência de mudanças e a falta de consciência arquitetural dos desenvolvedores estão entre as causas mais recorrentes desse processo (BAABAD et al., 2020). Esse quadro reforça a importância de abordagens arquiteturais que tornem explícitas as regras de organização e de dependência do código, de modo que possam ser compreendidas, aplicadas e verificadas ao longo do desenvolvimento.

Nesse cenário, a Clean Architecture, proposta por Martin (2017), apresenta-se como uma abordagem que organiza o software em camadas bem definidas, nas quais as regras de negócio são posicionadas no núcleo do sistema e permanecem isoladas de detalhes externos. O princípio fundamental dessa arquitetura é a regra de dependência, segundo a qual as dependências de código devem sempre apontar para as camadas mais internas, garantindo que mudanças tecnológicas não comprometam a lógica essencial da aplicação. A proposta não surge isolada: ela consolida características comuns a abordagens anteriores, como a arquitetura em camadas, a arquitetura hexagonal e a arquitetura cebola, que compartilham o objetivo de separar responsabilidades e proteger as regras de negócio de detalhes externos (MARTIN, 2017).

No recorte tecnológico deste trabalho, esse debate é situado no ecossistema JavaScript de servidor. O Node.js consolidou-se como uma das plataformas mais utilizadas para o desenvolvimento de APIs e serviços backend, e a linguagem TypeScript agrega tipagem estática a esse ecossistema, favorecendo a definição de contratos explícitos entre componentes (BIERMAN; ABADI; TORGERSEN, 2014). Essa combinação, amplamente adotada na indústria, constitui um cenário representativo para investigar a aplicação prática de princípios arquiteturais em projetos reais.

1.2 PROBLEMA

Apesar da ampla adoção de frameworks modernos no desenvolvimento backend, muitos sistemas ainda são construídos sem uma definição arquitetural clara, resultando em soluções fortemente acopladas e de difícil manutenção. A ausência de uma separação adequada entre

domínio e infraestrutura contribui para o aumento da complexidade estrutural e dificulta a evolução do software ao longo do tempo (FOWLER, 2002).

Esse problema é agravado pelo modo como muitos frameworks estruturam os projetos por padrão, organizando o código em torno de conceitos da própria ferramenta, como controladores, modelos e módulos, e não em torno das regras de negócio. Nessa organização, é comum que regras importantes do domínio fiquem dispersas entre camadas de apresentação e de persistência, o que dificulta sua identificação, seu teste e sua reutilização. As evidências sobre degradação arquitetural sugerem que, na ausência de regras explícitas de dependência e de mecanismos que favoreçam sua observância, o afastamento entre a estrutura pretendida e a estrutura implementada tende a se acentuar com a evolução do sistema (PERRY; WOLF, 1992; BAABAD et al., 2020).

Diante desse quadro, o problema investigado neste trabalho pode ser sintetizado na seguinte questão: como aplicar, de forma sistemática e demonstrável, os princípios da Clean Architecture em projetos backend desenvolvidos com Node.js e TypeScript, preservando a regra de dependência e o isolamento das regras de negócio em relação a frameworks, mecanismos de persistência e protocolos de comunicação?

O problema é delimitado ao desenvolvimento de APIs backend com Node.js e TypeScript, apoiadas em frameworks e ferramentas representativos desse ecossistema. Não estão no escopo deste trabalho a comparação quantitativa entre arquiteturas alternativas nem a avaliação de desempenho em ambiente de produção, aspectos que constituem oportunidades para investigações futuras.

1.3 JUSTIFICATIVA

Este trabalho justifica-se pela necessidade de apresentar não apenas os fundamentos conceituais da Clean Architecture, mas também sua aplicação prática em um projeto real, contribuindo para a compreensão, adoção e disseminação dessa abordagem no desenvolvimento de sistemas backend mais organizados, desacoplados e sustentáveis. A sistematização dessa experiência em formato de manual amplia o alcance da contribuição, ao oferecer a estudantes e desenvolvedores um material de referência que relaciona conceitos teóricos a decisões concretas de implementação.

Do ponto de vista acadêmico, a literatura sobre a Clean Architecture concentra-se, em grande parte, na apresentação de princípios e diagramas conceituais, com menor

disponibilidade de materiais que conectem esses princípios a decisões concretas de código em tecnologias específicas. Ao documentar uma implementação completa, camada por camada, acompanhada da análise das decisões tomadas, das boas práticas observadas e dos erros a evitar, este trabalho contribui para reduzir a distância entre a discussão conceitual e a prática de desenvolvimento.

Do ponto de vista prático e educacional, um manual estruturado pode apoiar tanto equipes de desenvolvimento interessadas em adotar a abordagem quanto estudantes em disciplinas de engenharia de software e de arquitetura de software. Considerando que parcela relevante dos custos de um sistema decorre de manutenção e evolução, e que a dívida técnica acumulada tende a ampliar esses custos (KRUCHTEN; NORD; OZKAYA, 2012), iniciativas que favoreçam a construção de sistemas mais organizados e desacoplados possuem potencial de impacto direto sobre a sustentabilidade do software.

1.4 MOTIVAÇÃO

A motivação para este trabalho decorre da observação de que, embora a Clean Architecture seja amplamente discutida na literatura, sua aplicação prática ainda representa um desafio para muitos desenvolvedores, especialmente em contextos reais que envolvem tecnologias modernas e frameworks populares. Nesse sentido, torna-se relevante investigar os princípios da Clean Architecture e demonstrar sua aplicação prática em um projeto backend, com foco na organização estrutural do código e na independência entre camadas, aproximando a discussão conceitual da prática cotidiana de desenvolvimento.

Soma-se a isso a observação de que os materiais disponíveis sobre o tema, em geral, apresentam exemplos fragmentados ou desvinculados de um projeto completo, o que dificulta a compreensão de como os princípios se articulam em um sistema real. A possibilidade de construir uma aplicação de ponta a ponta, documentando as decisões de cada camada e os erros mais comuns em cada etapa, motivou a escolha do formato de manual como produto central deste trabalho.

1.5 OBJETIVOS

1.5.1 Objetivo Geral

Propor um manual de implementação da Clean Architecture em projetos Node.js com TypeScript, demonstrado por meio do desenvolvimento de uma API RESTful, discutindo as implicações da aplicação de seus princípios para a organização, manutenibilidade e evolução do software.

1.5.2 Objetivos Específicos

- Compreender os conceitos fundamentais de arquitetura de software e Clean Architecture;
- Estruturar uma aplicação backend em camadas bem definidas, respeitando a regra de dependência;
- Implementar entidades, casos de uso, interfaces e adaptadores de acordo com os princípios da Clean Architecture;
- Demonstrar, por meio de código, a separação entre domínio e detalhes de infraestrutura;
- Sistematizar boas práticas e erros frequentes observados na implementação de cada camada, em formato de manual;
- Analisar os benefícios da aplicação da Clean Architecture em termos de organização e manutenibilidade do sistema.

1.6 METODOLOGIA

Este trabalho caracteriza-se como uma pesquisa de natureza aplicada, com abordagem qualitativa e caráter exploratório-descritivo. A natureza aplicada justifica-se pelo desenvolvimento de um artefato tecnológico voltado à demonstração prática de um modelo arquitetural, enquanto a abordagem qualitativa permite analisar aspectos estruturais e conceituais relacionados à organização do software e à aplicação de princípios arquiteturais (GIL, 2022; CRESWELL; CRESWELL, 2018).

Do ponto de vista metodológico, o estudo adota uma estratégia baseada na construção e na análise de um artefato de software, alinhando-se aos princípios da Design Science Research, que orienta a produção de conhecimento científico por meio da construção e da avaliação de artefatos (HEVNER et al., 2004; WIERINGA, 2014). Sob esse enfoque, a aplicação

desenvolvida não é apenas um produto funcional, mas um meio para investigar a aplicação prática dos princípios da Clean Architecture em um ambiente controlado.

A condução do trabalho foi organizada em etapas sucessivas, sintetizadas na Figura 1. Inicialmente, realizou-se o levantamento e a análise dos conceitos fundamentais de arquitetura de software e da Clean Architecture, com base em literatura especializada. Em seguida, procedeu-se à modelagem da aplicação, com a definição das camadas arquiteturais, das responsabilidades dos componentes e dos contratos entre interfaces. Na sequência, implementou-se o sistema de gerenciamento de listas de tarefas, estruturado em entidades, casos de uso, interfaces, adaptadores de apresentação e infraestrutura, em conformidade com a regra de dependência.

Figura 1 – Etapas metodológicas da pesquisa



Fonte: elaborado pelos autores (2026).

Durante a implementação, trechos de código foram utilizados como evidências empíricas da aplicação dos conceitos teóricos. Esses trechos são apresentados e analisados ao longo do trabalho, o que permite estabelecer uma relação direta entre os princípios arquiteturais e sua materialização no sistema desenvolvido.

Posteriormente, o artefato foi avaliado no Capítulo 5, de forma qualitativa, segundo critérios derivados da própria literatura de arquitetura de software: independência do domínio em relação a tecnologias externas, clareza estrutural, inversão de dependência, testabilidade, isolamento das preocupações de borda e esforço de adoção. Essa avaliação, apoiada em análise argumentativa e na inspeção do código produzido, é compatível com a Design Science Research, que admite a demonstração da utilidade do artefato por meio de cenários e de argumentação fundamentada (HEVNER et al., 2004).

Por fim, sistematizaram-se as boas práticas e os equívocos recorrentes identificados ao longo da construção do artefato, consolidando-os no formato de um manual de implementação. Essa etapa constitui a contribuição central deste trabalho, ao reunir, de maneira organizada e progressiva, as orientações para a aplicação da Clean Architecture em projetos Node.js com TypeScript.

1.7 ESTRUTURA DO TRABALHO

Este trabalho está organizado em seis capítulos. O Capítulo 1 apresenta a introdução, contemplando contextualização, problema, justificativa, motivação, objetivos, metodologia e estrutura do trabalho. O Capítulo 2 apresenta a fundamentação teórica, abordando conceitos de arquitetura de software e Clean Architecture. O Capítulo 3 descreve a aplicação prática desenvolvida e as tecnologias adotadas. O Capítulo 4 apresenta o manual de implementação da Clean Architecture, detalhando a aplicação dos conceitos por meio da análise das camadas do sistema. O Capítulo 5 discute os resultados obtidos a partir da aplicação da arquitetura proposta. Por fim, o Capítulo 6 apresenta as considerações finais e sugestões para trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 ARQUITETURA DE SOFTWARE

A arquitetura de software pode ser definida como a estrutura fundamental de um sistema, composta pelos seus elementos, pelas relações existentes entre eles e pelos princípios que orientam seu projeto e evolução. Segundo Bass, Clements e Kazman (2021), a arquitetura representa um conjunto de decisões estruturais que impactam diretamente atributos de qualidade como desempenho, manutenibilidade, segurança e escalabilidade.

Em um dos trabalhos seminais da área, Perry e Wolf (1992) caracterizam a arquitetura de software a partir de elementos de processamento, elementos de dados e elementos de conexão, acompanhados de princípios e propriedades que restringem como esses elementos podem ser combinados. Essa caracterização evidencia que a arquitetura não se resume ao desenho de módulos: ela inclui as razões por trás das decisões estruturais, o que a torna um instrumento de comunicação e de análise ao longo de todo o ciclo de vida do sistema.

De forma semelhante, Pressman e Maxim (2021) afirmam que a arquitetura de software fornece uma visão de alto nível do sistema, permitindo compreender como seus componentes estão organizados e como interagem para atender aos requisitos funcionais e não funcionais. Essa organização é essencial para controlar a complexidade inerente ao desenvolvimento de sistemas modernos.

Os atributos de qualidade influenciados pela arquitetura são sistematizados pela norma ISO/IEC 25010, que define um modelo de qualidade de produto composto por características como adequação funcional, eficiência de desempenho, confiabilidade, segurança e manutenibilidade (ISO, 2023). Para os fins deste trabalho, destaca-se a manutenibilidade, definida como o grau de eficácia e de eficiência com que um produto pode ser modificado, e desdobrada em subcaracterísticas como modularidade, reusabilidade, analisabilidade, modificabilidade e testabilidade.

Essas subcaracterísticas oferecem um vocabulário preciso para discutir os efeitos de decisões arquiteturais: a modularidade trata do grau em que o sistema é composto por componentes discretos, de forma que a mudança em um deles tenha impacto mínimo nos demais; a modificabilidade trata da capacidade de alterar o sistema sem introduzir defeitos; e a testabilidade trata da facilidade de estabelecer critérios de teste e de verificá-los. São justamente esses os atributos retomados como critérios de avaliação do artefato no Capítulo 5.

A definição de uma arquitetura adequada influencia diretamente a capacidade de evolução do software ao longo do tempo. Sistemas desenvolvidos sem uma estrutura arquitetural clara tendem a apresentar maior dificuldade de manutenção, além de elevado acoplamento entre componentes. Em contrapartida, arquiteturas bem definidas favorecem a separação de responsabilidades, a reutilização de componentes e a redução do impacto causado por mudanças futuras (BASS; CLEMENTS; KAZMAN, 2021).

Dessa forma, a arquitetura de software não deve ser vista apenas como uma representação estrutural do sistema, mas como um mecanismo estratégico para garantir sua sustentabilidade e qualidade ao longo de todo o ciclo de vida.

Cabe destacar, ainda, que a arquitetura planejada só produz os benefícios esperados se estiver refletida no código efetivamente implementado. Como discutido no Capítulo 1, a distância entre a estrutura pretendida e a estrutura realizada tende a crescer com a evolução do sistema, caracterizando a erosão arquitetural (PERRY; WOLF, 1992; BAABAD et al., 2020). Abordagens como a Clean Architecture procuram mitigar esse risco ao expressar as regras estruturais em convenções simples e verificáveis, como a direção permitida das dependências entre as camadas.

2.2 PROBLEMAS DE ACOPLAMENTO

Um dos principais desafios enfrentados no desenvolvimento de software é o acoplamento excessivo entre componentes. O acoplamento pode ser compreendido como o grau de dependência existente entre diferentes partes de um sistema. Quando um componente depende fortemente da implementação de outro, alterações em uma parte da aplicação tendem a provocar impactos inesperados em diversos pontos do sistema.

As noções de acoplamento e coesão remontam aos trabalhos sobre projeto estruturado de Stevens, Myers e Constantine (1974), nos quais o acoplamento mede o grau de interdependência entre módulos e a coesão mede o quanto as responsabilidades internas de um módulo estão relacionadas entre si. Desde então, a combinação de baixo acoplamento com alta coesão consolidou-se como um dos principais critérios de qualidade de projeto, por facilitar a compreensão, o teste e a modificação de cada parte do sistema de forma isolada.

Segundo Fowler (2002), sistemas fortemente acoplados tornam-se mais difíceis de compreender, testar e modificar, uma vez que mudanças aparentemente simples podem exigir

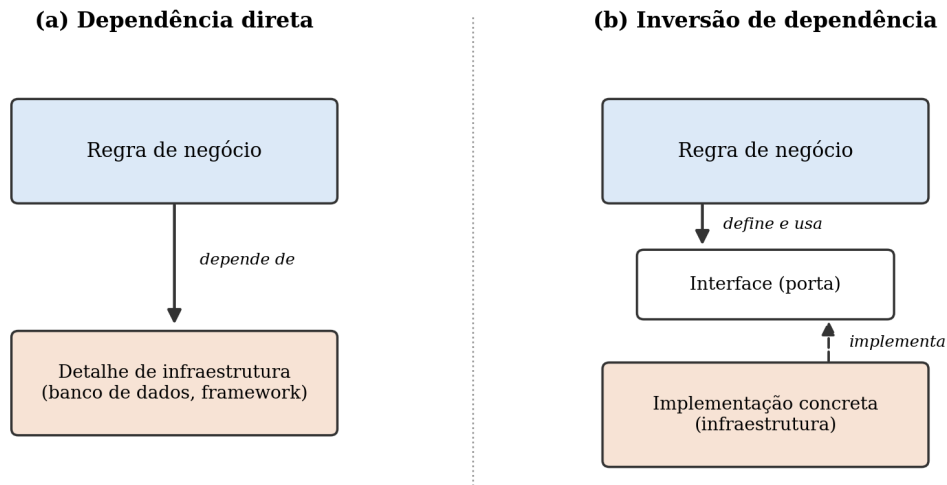
alterações em múltiplos componentes. Esse cenário aumenta o custo de manutenção e reduz a capacidade de evolução do software.

O acoplamento excessivo também dificulta a substituição de tecnologias e a adoção de novas soluções. Quando regras de negócio estão diretamente associadas a frameworks, bancos de dados ou mecanismos específicos de infraestrutura, qualquer mudança tecnológica pode exigir modificações significativas no núcleo da aplicação.

Estudos recentes indicam que parte desse acoplamento é induzida pelas próprias tecnologias adotadas: frameworks e plataformas pressionam os desenvolvedores a tomar decisões de projeto que não tomariam em um cenário ideal, aproximando o código de estruturas e convenções da ferramenta em vez de estruturas do domínio (ANICHE; YODER; KON, 2019). Esse efeito reforça a necessidade de mecanismos arquiteturais que delimitem explicitamente o alcance dos frameworks dentro do sistema.

Por esse motivo, arquiteturas modernas buscam reduzir o acoplamento por meio da separação de responsabilidades e da utilização de abstrações. Essa abordagem permite que diferentes partes do sistema evoluam de forma mais independente, favorecendo a manutenibilidade e a longevidade do software.

Entre esses mecanismos, destaca-se o princípio da inversão de dependência, segundo o qual módulos de alto nível não devem depender de módulos de baixo nível: ambos devem depender de abstrações (MARTIN, 2017). Na prática, isso significa que a regra de negócio define o contrato de que necessita, na forma de uma interface, e a infraestrutura fornece a implementação desse contrato, invertendo a direção tradicional da dependência. A Figura 2 ilustra a diferença entre a dependência direta da regra de negócio em relação a detalhes de infraestrutura e a configuração obtida com a inversão de dependência.

Figura 2 – Dependência direta e inversão de dependência

Fonte: elaborado pelos autores (2026).

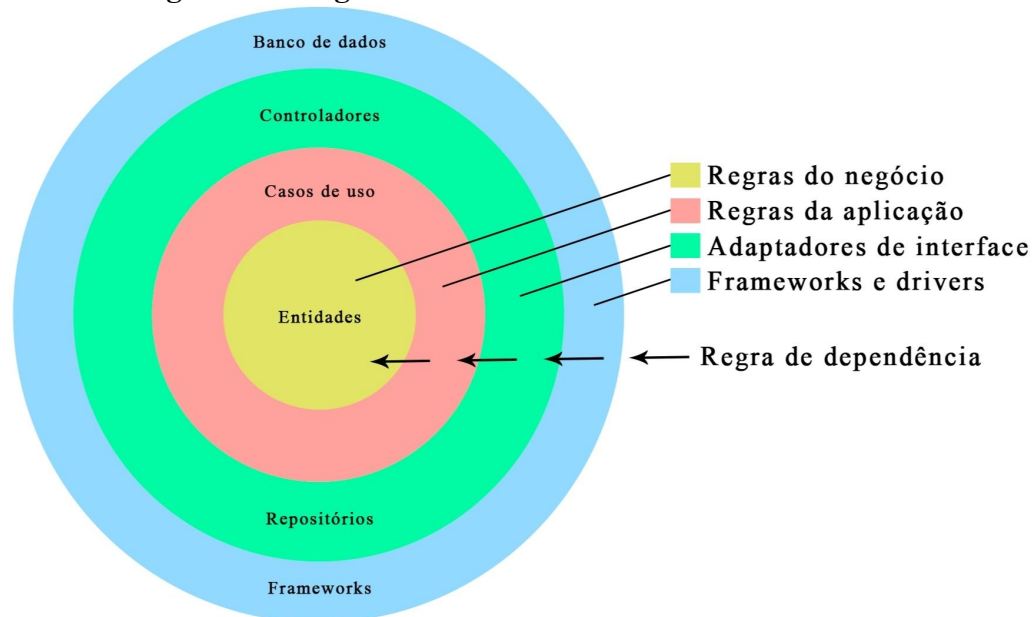
2.3 CLEAN ARCHITECTURE

A Clean Architecture é uma proposta arquitetural apresentada por Robert C. Martin com o objetivo de organizar sistemas de software de forma que as regras de negócio permaneçam protegidas contra mudanças em tecnologias externas. A arquitetura é baseada na separação de responsabilidades em camadas, permitindo que o núcleo da aplicação permaneça independente de frameworks, bancos de dados e interfaces de usuário (MARTIN, 2017).

Segundo Martin (2017), a principal característica da Clean Architecture é a organização do software em camadas concêntricas, nas quais os elementos mais internos representam as regras de negócio e os elementos mais externos representam mecanismos de implementação.

Essa estrutura busca garantir que decisões de negócio permaneçam estáveis mesmo diante de alterações tecnológicas.

No modelo de referência, quatro camadas são apresentadas. No centro estão as entidades, que concentram as regras de negócio corporativas. Ao redor delas ficam os casos de uso, que implementam as regras específicas da aplicação. A camada seguinte reúne os adaptadores de interface, como controladores, apresentadores e gateways, responsáveis pela conversão de dados entre os formatos internos e externos. Por último, a camada mais externa abriga frameworks e drivers, como bancos de dados, servidores web e demais ferramentas (MARTIN, 2017). A Figura 3 apresenta essa organização em camadas concêntricas, com as dependências apontando para o centro.

Figura 3 – Diagrama de camadas da Clean Architecture

Fonte: adaptado de Martin (2017).

Um dos princípios centrais dessa arquitetura é a chamada regra de dependência. De acordo com Martin (2017), as dependências de código devem sempre apontar para as camadas mais internas da aplicação. Isso significa que elementos externos podem conhecer componentes internos, mas componentes internos não devem conhecer detalhes de implementação das camadas externas.

Para que os fluxos de execução atravessassem as fronteiras entre camadas sem violar a regra de dependência, a Clean Architecture recorre à inversão de dependência nas fronteiras: as camadas internas declaram interfaces e as camadas externas as implementam (MARTIN, 2017). Além disso, os dados que cruzam as fronteiras devem ser estruturas simples, evitando que tipos de frameworks ou registros de banco de dados penetrem nas camadas internas.

A aplicação dessa regra permite que frameworks, mecanismos de persistência e protocolos de comunicação sejam tratados como detalhes substituíveis. Desse modo, o núcleo do sistema permanece protegido contra mudanças tecnológicas, favorecendo a manutenibilidade e a evolução do software ao longo do tempo.

A adoção da Clean Architecture, contudo, não é isenta de custos. A criação de interfaces, modelos de dados intermediários e mecanismos de composição adiciona indireções e aumenta o volume inicial de código, o que pode parecer desproporcional em aplicações de pequeno porte (MARTIN, 2017). A literatura aplicada sugere que esses custos tendem a ser compensados em sistemas com expectativa de evolução, nos quais os ganhos de manutenibilidade e de

testabilidade se acumulam ao longo do tempo (NUGROHO; KUSUMO; ALIBASA, 2022). Essa relação entre esforço inicial e benefícios é retomada na avaliação conduzida no Capítulo 5.

2.4 CONCEITOS-CHAVE DA CLEAN ARCHITECTURE

Para compreender a aplicação prática da Clean Architecture, é necessário conhecer os principais componentes que compõem sua estrutura. Entre eles destacam-se as entidades, os casos de uso, as interfaces e os adaptadores, elementos que serão explorados em maior profundidade no Capítulo 4.

2.4.1 Entidades

As entidades representam as regras de negócio mais gerais e estáveis do sistema. Segundo Martin (2017), elas encapsulam os conceitos fundamentais do domínio e devem permanecer independentes de frameworks, bancos de dados e demais detalhes externos. Por ocuparem a camada mais interna da arquitetura, as entidades constituem o núcleo do software e tendem a sofrer poucas alterações ao longo do tempo.

Esse conceito guarda relação direta com a noção de entidade do Domain-Driven Design, na qual objetos do domínio possuem identidade própria e ciclo de vida independente de sua representação em mecanismos de persistência (EVANS, 2003). Em conjunto com os objetos de valor, as entidades formam o vocabulário central com o qual as regras de negócio são expressas e validadas.

Cabe distinguir, nesse contexto, entidades de objetos de valor. Enquanto a entidade é definida por sua identidade, que se mantém ao longo do tempo mesmo quando seus atributos mudam, o objeto de valor é definido exclusivamente pelos valores que carrega: dois objetos de valor com os mesmos atributos são considerados iguais e, por isso, podem ser tratados como imutáveis e livremente substituíveis (EVANS, 2003). Identificadores, datas e quantias monetárias são exemplos típicos. No estudo de caso deste trabalho, essa distinção se materializa no objeto de valor que representa identificadores, analisado no Capítulo 4.

2.4.2 Casos de Uso

Os casos de uso representam as regras específicas da aplicação e descrevem como o sistema deve responder às ações executadas pelos usuários ou por sistemas externos. De acordo com Martin (2017), essa camada é responsável por coordenar entidades e abstrações para executar fluxos de negócio, mantendo-se independente de detalhes tecnológicos.

Na prática, cada caso de uso recebe uma solicitação, recupera ou constrói as entidades necessárias, aplica as regras pertinentes e devolve uma resposta, frequentemente mediante modelos de entrada e de saída próprios. Essa camada conhece as entidades, mas não conhece os mecanismos que originam as solicitações nem os destinos das respostas, o que preserva sua independência em relação à interface com o usuário e à infraestrutura (MARTIN, 2017).

2.4.3 Interfaces

As interfaces, também chamadas de portas na Clean Architecture, definem contratos de comunicação entre as camadas internas e externas da aplicação. Segundo Martin (2017), elas permitem que a lógica de negócio dependa de abstrações em vez de implementações concretas, contribuindo para a redução do acoplamento e para a aplicação do princípio da inversão de dependência.

Ao depender de interfaces, os casos de uso podem ser exercitados em testes com implementações simuladas, sem a presença de banco de dados ou de servidores reais. Além disso, a substituição de uma tecnologia de persistência ou de comunicação passa a exigir apenas uma nova implementação do contrato, sem alterações nas camadas internas da aplicação.

Na prática, a inversão de dependência costuma ser operacionalizada pelo mecanismo de injeção de dependência, no qual as implementações concretas são fornecidas aos componentes que delas dependem por um agente externo, em vez de instanciadas internamente. Frameworks modernos oferecem contêineres de injeção que resolvem essas associações em um ponto único de composição da aplicação, mantendo o restante do código orientado a contratos (MARTIN, 2017). Esse mecanismo é explorado no Capítulo 4, na composição dos módulos da aplicação desenvolvida.

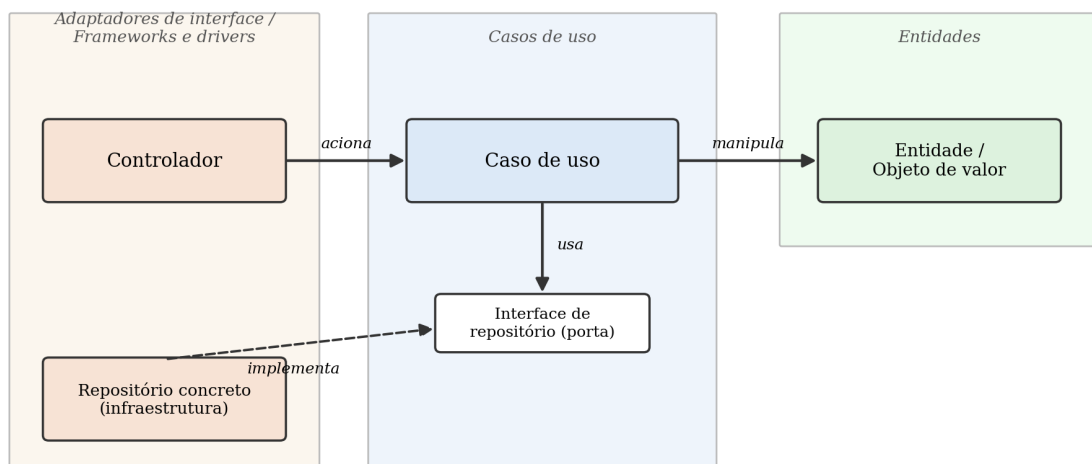
2.4.4 Adaptadores

Os adaptadores são responsáveis por converter informações entre o formato utilizado pelas camadas externas e o formato esperado pelas camadas internas da aplicação. Martin

(2017) descreve essa camada como um mecanismo de tradução que permite a comunicação entre interfaces externas, como APIs e bancos de dados, e os componentes centrais do sistema.

Controladores, apresentadores e gateways são exemplos típicos dessa camada. A Figura 4 ilustra o fluxo de uma requisição através das camadas: o controlador recebe a solicitação externa e aciona o caso de uso, que manipula entidades e acessa a persistência por meio de uma interface implementada na infraestrutura. Nesse arranjo, todas as dependências de código continuam apontando para o interior da arquitetura, ainda que o fluxo de controle atravesse as fronteiras nos dois sentidos.

Figura 4 – Fluxo de uma requisição através das camadas da Clean Architecture



Fonte: elaborado pelos autores (2026).

2.5 PRINCÍPIOS E PADRÕES DE APOIO

A Clean Architecture não opera isolada: ela se apoia em princípios de projeto e em padrões consolidados que orientam a implementação de cada camada. Esta seção apresenta os princípios SOLID e os padrões de persistência e de transferência de dados utilizados no manual desenvolvido neste trabalho.

2.5.1 Princípios SOLID

O acrônimo SOLID reúne cinco princípios de projeto orientado a objetos sistematizados por Martin (2017): responsabilidade única, aberto-fechado, substituição de Liskov, segregação de interfaces e inversão de dependência. O princípio da responsabilidade única estabelece que

cada módulo deve ter um único motivo para mudar, o que se traduz, na arquitetura adotada, em casos de uso enxutos e dedicados a um único fluxo de negócio. O princípio aberto-fechado, por sua vez, orienta que o comportamento do sistema seja extensível sem a modificação do código existente, favorecendo a adição de novos fluxos pela criação de novos componentes.

O princípio da substituição de Liskov exige que implementações de um mesmo contrato sejam intercambiáveis sem quebra de comportamento, condição necessária para que repositórios concretos possam substituir uns aos outros sob a mesma interface. A segregação de interfaces recomenda contratos pequenos e específicos, evitando que componentes dependam de operações que não utilizam. Finalmente, a inversão de dependência, já discutida na Seção 2.2, é o princípio que sustenta diretamente a regra de dependência da Clean Architecture, ao determinar que políticas de alto nível e detalhes de baixo nível dependam ambos de abstrações (MARTIN, 2017). No Capítulo 4, esses princípios reaparecem nas boas práticas apontadas em cada camada do sistema.

2.5.2 Padrões de persistência e de transferência de dados

Entre os padrões de arquitetura de aplicações corporativas catalogados por Fowler (2002), o padrão Repository ocupa papel central na arquitetura adotada. Um repositório media entre o domínio e o mecanismo de persistência, oferecendo uma interface semelhante a uma coleção em memória para o acesso a objetos do domínio. Na Clean Architecture, o contrato do repositório é declarado junto às camadas internas e implementado na infraestrutura, o que permite que casos de uso recuperem e persistam entidades sem qualquer conhecimento sobre bancos de dados, consultas ou mapeamentos (FOWLER, 2002; EVANS, 2003).

De forma complementar, a travessia das fronteiras entre camadas apoia-se em objetos de transferência de dados, estruturas simples que carregam informações entre processos ou camadas sem expor entidades do domínio nem registros do banco de dados (FOWLER, 2002). O uso desses objetos nas entradas e saídas dos casos de uso e nas respostas dos controladores evita que tipos externos penetrem nas camadas internas, preservando a regra de dependência. Ambos os padrões são aplicados e analisados no Capítulo 4.

2.6 TRABALHOS RELACIONADOS

Esta seção apresenta trabalhos que investigam a aplicação da Clean Architecture em diferentes contextos tecnológicos, com o objetivo de situar a contribuição deste estudo em relação às investigações recentes sobre o tema.

Beltrão, Farzat e Travassos (2020) relatam a experiência de uma fábrica de software brasileira na institucionalização da Clean Architecture como instrumento de gestão da dívida técnica em um projeto industrial. Partindo da constatação de que as ferramentas de análise estática usuais oferecem apenas regras agnósticas em relação à arquitetura, os autores definiram um conjunto inicial de 52 regras de verificação derivadas do modelo de Martin (2017) e de convenções do próprio cliente, associando cada regra a uma camada específica da arquitetura, como domínio, casos de uso, repositórios e serviços externos.

Essas regras foram implementadas como um complemento para a plataforma SonarQube e integradas ao ciclo de desenvolvimento, permitindo acompanhar a dívida técnica em nível de implementação ao longo do tempo. Os resultados preliminares indicam que a formalização de regras arquiteturais torna a regra de dependência verificável de forma automatizada, transformando a conformidade arquitetural em um atributo mensurável do processo de desenvolvimento.

Nugroho, Kusumo e Alibasa (2022) investigam os impactos da adoção da Clean Architecture sobre a manutenibilidade da base de código de um sistema backend. Os autores partem de um código com problemas recorrentes nesse tipo de sistema, como a duplicação extensa de trechos em diferentes pontos da aplicação, e o refatoram em camadas organizadas segundo a hierarquia da lógica de negócio, em conformidade com a regra de dependência.

A avaliação compara a base de código antes e depois da refatoração por intermédio de métricas consolidadas na literatura, incluindo complexidade ciclomática, contagem ponderada de métodos, estimativas de defeitos, medidas de Halstead e o índice de manutenibilidade. Os resultados indicam melhora em todas as métricas analisadas após a aplicação dos princípios da Clean Architecture, oferecendo indícios quantitativos de que os benefícios atribuídos à abordagem se manifestam em sistemas reais.

Brown et al. (2021) descrevem a refatoração do componente de servidor de um sistema tutor inteligente voltado ao ensino de análise de segurança de sistemas, originalmente implementado em Prolog com um sistema de módulos. Em busca de uma organização alinhada à Clean Architecture, os autores migraram a aplicação para a linguagem Logtalk e desacoplaram o motor de raciocínio, o arcabouço de autoria de ontologias e o mecanismo de suporte pedagógico em bibliotecas independentes, reutilizáveis em outras aplicações.

A inversão de dependências foi viabilizada por protocolos e categorias da linguagem, mecanismos equivalentes às interfaces utilizadas em ecossistemas orientados a objetos. As duas iterações da arquitetura são avaliadas quanto ao reúso de código e à extensibilidade, e os resultados demonstram que os princípios da Clean Architecture não se restringem a plataformas convencionais, podendo ser aplicados inclusive em ambientes de programação lógica.

Kaczmariski (2024) examina a implementação de componentes de comunicação web RESTful em C# com a versão 8 da plataforma .NET. O trabalho revisita as propriedades que caracterizam uma arquitetura de rede como RESTful, discute a coexistência do REST com tecnologias como GraphQL e gRPC e descreve padrões de implementação específicos da plataforma, comparando as abordagens de APIs mínimas e de controladores.

Na parte final, o autor discute e testa uma organização padronizada da API baseada na Clean Architecture, concluindo que a combinação oferece um ferramental adequado para o desenvolvimento de arquiteturas de comunicação eficientes e bem estruturadas. O estudo aproxima-se do presente trabalho por integrar o estilo REST à Clean Architecture, embora em outro ecossistema tecnológico e sem o caráter de manual didático adotado aqui.

Em síntese, os trabalhos relacionados evidenciam o interesse crescente na Clean Architecture e fornecem indícios de seus benefícios, mas concentram-se na verificação automatizada de regras, na avaliação de manutenibilidade ou em ecossistemas distintos do JavaScript de servidor. Este trabalho diferencia-se ao propor um manual de implementação voltado ao ecossistema Node.js com TypeScript, no qual cada camada é construída e analisada passo a passo, com a explicitação das decisões tomadas, das boas práticas observadas e dos erros a evitar. Essa orientação didática, apoiada em um sistema completo e funcional, complementa os estudos anteriores ao reduzir a distância entre os princípios arquiteturais e sua aplicação prática.

2.7 SÍNTESE DOS CONCEITOS

Os conceitos apresentados neste capítulo fornecem a base teórica necessária para compreender a Clean Architecture e seus principais componentes. Embora tenham sido apresentados de forma resumida, esses elementos constituem o fundamento da organização arquitetural proposta por Martin (2017). No Capítulo 3, esses conceitos são contextualizados na aplicação desenvolvida e, no Capítulo 4, são analisados sob uma perspectiva prática, por meio da implementação de uma API RESTful em Node.js e TypeScript, evidenciando como

cada camada da arquitetura foi aplicada no estudo de caso deste trabalho. Os trabalhos relacionados discutidos na Seção 2.6, por sua vez, situam a contribuição deste estudo no contexto das investigações recentes sobre o tema.

3 APLICAÇÃO PRÁTICA E TECNOLOGIAS ADOTADAS

Esta seção apresenta a aplicação prática desenvolvida neste trabalho, cujo objetivo é demonstrar, de forma concreta, a implementação da Clean Architecture em um cenário realista. Para isso, foi construída uma API RESTful para gerenciamento de listas de tarefas, utilizando Node.js e TypeScript, servindo como base para a materialização dos conceitos arquiteturais discutidos no capítulo anterior.

A escolha de uma aplicação prática, em vez da análise de um sistema existente, permite maior controle sobre as decisões arquiteturais adotadas, possibilitando a construção de um exemplo alinhado, desde sua concepção, aos princípios da Clean Architecture. Conforme destacado por Martin (2017), a clareza na separação de responsabilidades e na definição das camadas do sistema é fundamental para a preservação da lógica de negócio e para a independência de detalhes externos.

O sistema proposto foi deliberadamente escolhido por apresentar um domínio simples, porém suficientemente expressivo para ilustrar decisões arquiteturais relevantes, como separação de responsabilidades, definição de regras de negócio, aplicação da inversão de dependência e desacoplamento entre camadas. Essa escolha permite concentrar a análise nos aspectos estruturais da aplicação, evitando complexidades desnecessárias que poderiam obscurecer os princípios arquiteturais em foco.

Assim, a aplicação desenvolvida não deve ser interpretada apenas como um sistema funcional, mas como um artefato didático, projetado para evidenciar, por meio de código real, como os conceitos da Clean Architecture podem ser aplicados na prática em um ambiente backend moderno.

3.1 DESCRIÇÃO DO SISTEMA PROPOSTO

O sistema desenvolvido consiste em uma API de gerenciamento de listas de tarefas (To-Do List), na qual um usuário pode criar listas e, dentro de cada lista, gerenciar tarefas associadas. Cada lista representa um agrupamento lógico de tarefas, permitindo uma organização mais estruturada das atividades a serem realizadas.

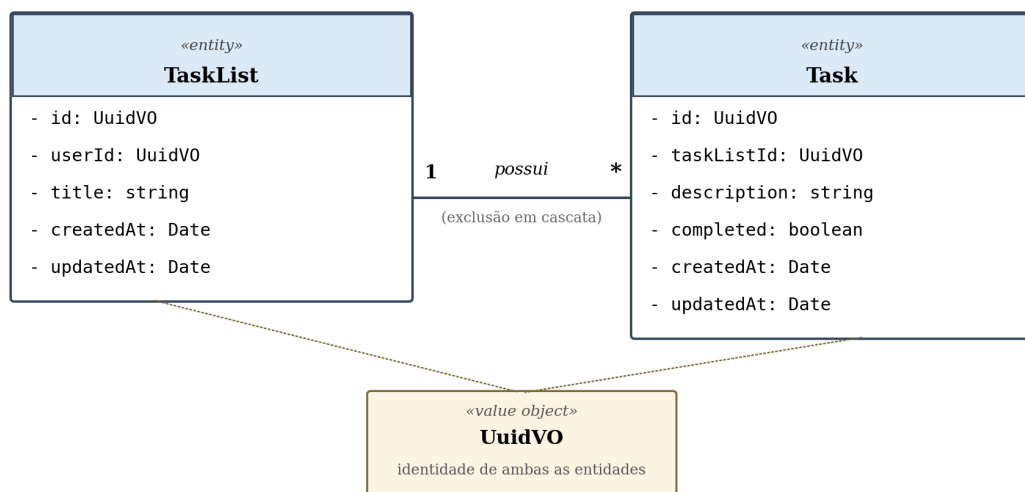
As tarefas possuem informações básicas, como descrição, estado de conclusão e associação a uma lista específica, enquanto cada lista possui um título. O sistema permite que uma tarefa seja criada, atualizada, marcada como concluída ou não concluída, listada e

removida. Da mesma forma, as listas de tarefas podem ser criadas, atualizadas, listadas e excluídas, respeitando sempre o escopo do usuário responsável.

O foco do sistema não está na complexidade das regras de negócio, mas sim na forma como essas regras são organizadas e protegidas por uma Clean Architecture. Essa escolha está alinhada com o objetivo do trabalho de propor um manual de implementação da Clean Architecture em projetos Node.js com TypeScript, utilizando a aplicação desenvolvida como demonstração prática de seus princípios e como base para a discussão de suas implicações para a organização, manutenibilidade e evolução do software.

A Figura 5 exibe o modelo de domínio do sistema. Cada lista de tarefas, representada pela entidade TaskList, está associada a um usuário e agrupa diversas tarefas, representadas pela entidade Task. Ambas as entidades utilizam o objeto de valor UuidVO como identidade, e a relação entre lista e tarefas adota exclusão em cascata, de maneira que a remoção de uma lista implica a remoção das tarefas a ela associadas.

Figura 5 – Modelo de domínio do sistema



Fonte: elaborado pelos autores (2026).

3.2 FUNCIONALIDADES DO SISTEMA

Do ponto de vista funcional, a API disponibiliza dois grupos de operações. O primeiro grupo concentra o gerenciamento de listas de tarefas e contempla a criação de uma nova lista, a atualização de seu título, a listagem de todas as listas pertencentes ao usuário identificado na requisição e a exclusão de uma lista, com a consequente remoção das tarefas a ela associadas.

O segundo grupo concentra o gerenciamento de tarefas no escopo de uma lista específica e contempla a criação de uma tarefa com descrição obrigatória, a atualização de seus dados, a alternância do estado de conclusão, a listagem das tarefas pertencentes à lista e a remoção de uma tarefa. Todas as operações são executadas no contexto do usuário identificado na requisição, de modo que um usuário não acessa nem modifica listas e tarefas de outros usuários. Esse recorte funcional, embora enxuto, é suficiente para exercitar os três tipos de fluxo explorados no manual: criação de entidades, manipulação de estado de entidades existentes e operações de consulta.

Do ponto de vista arquitetural, o sistema foi projetado para permitir a aplicação explícita dos princípios da Clean Architecture, especialmente no que se refere à separação de responsabilidades, isolamento do domínio e inversão de dependência. Assim, a lógica de negócio foi estruturada de forma independente de detalhes externos, como mecanismos de persistência, frameworks e protocolos de comunicação.

3.3 CONTEXTO DE USUÁRIO E IDENTIFICAÇÃO POR X-USER-ID

Para o correto funcionamento do sistema, é necessário identificar o usuário responsável pelas listas e tarefas manipuladas, uma vez que todas as operações realizadas na aplicação estão associadas a um contexto específico de usuário. Essa identificação é essencial para garantir o isolamento dos dados e a consistência das operações no domínio da aplicação.

Entretanto, o desenvolvimento de um mecanismo completo de autenticação e autorização, envolvendo funcionalidades como login, gerenciamento de credenciais e uso de tokens JWT, não faz parte do escopo deste trabalho. Essa decisão foi tomada considerando que o foco principal está na organização arquitetural do sistema, e não na implementação de mecanismos de segurança.

Com isso, foi adotada uma abordagem simplificada para identificação do usuário, baseada no envio de um identificador único por meio do cabeçalho HTTP `x-user-id`. Esse identificador representa um usuário previamente autenticado e autorizado por um sistema externo, que não é implementado neste projeto. Assim, a API assume que o processo de autenticação e autorização já foi realizado antes da requisição atingir a aplicação, delegando essa responsabilidade a um componente externo.

Do ponto de vista arquitetural, essa decisão permite simular um cenário realista de integração entre serviços, no qual a API de gerenciamento de tarefas atua de forma

independente, concentrando-se exclusivamente em suas regras de negócio. Além disso, evita o acoplamento entre a lógica de autenticação e o domínio da aplicação, preservando a separação de responsabilidades.

Essa abordagem está alinhada com os princípios da Clean Architecture, que recomendam o isolamento do domínio em relação a detalhes externos, como frameworks, protocolos e serviços de autenticação. Ao tratar a identificação do usuário como um dado de entrada proveniente da camada de apresentação, o sistema mantém sua lógica central independente de mecanismos específicos de segurança, respeitando a regra de dependência proposta por Martin (2017).

3.4 ARQUITETURA DA API

A API foi desenvolvida seguindo o estilo arquitetural REST (FIELDING, 2000), no qual os recursos do sistema são expostos mediante endpoints bem definidos e manipulados utilizando métodos HTTP apropriados, como GET, POST, PUT e DELETE. As requisições e respostas utilizam o formato JSON, amplamente adotado em aplicações web, o que facilita a interoperabilidade entre sistemas heterogêneos e contribui para a simplicidade na troca de dados.

A Figura 6 relaciona os endpoints expostos pela API, o método HTTP correspondente, a operação realizada e o caso de uso acionado em cada situação. As operações sobre listas são acessadas a partir do recurso de listas, enquanto as operações de criação e listagem de tarefas são aninhadas sob a lista correspondente, e as operações sobre uma tarefa específica são acessadas diretamente pelo identificador da tarefa.

Figura 6 – Endpoints da API e casos de uso correspondentes

Método	Rota	Operação	Caso de uso
GET	/task-lists	Listar as listas do usuário	GetTaskListsUseCase
POST	/task-lists	Criar uma lista	CreateTaskListUseCase
PUT	/task-lists/:id	Atualizar o título de uma lista	UpdateTaskListUseCase
DELETE	/task-lists/:id	Excluir uma lista e suas tarefas	DeleteTaskListUseCase
GET	/task-lists/:listId/tasks	Listar as tarefas de uma lista	GetTasksByListUseCase
POST	/task-lists/:listId/tasks	Criar uma tarefa em uma lista	CreateTaskUseCase
PUT	/tasks/:id	Atualizar uma tarefa	UpdateTaskUseCase
PATCH	/tasks/:id/toggle	Alternar a conclusão de uma tarefa	ToggleTaskCompletionUseCase
DELETE	/tasks/:id	Excluir uma tarefa	DeleteTaskUseCase

Fonte: elaborado pelos autores (2026).

Do ponto de vista arquitetural, a aplicação foi estruturada de modo a viabilizar a implementação da Clean Architecture, promovendo a separação clara entre domínio, aplicação e detalhes externos. Essa organização garante que as regras de negócio permaneçam isoladas de elementos como frameworks, mecanismos de persistência e protocolos de comunicação, reduzindo o acoplamento entre componentes e favorecendo a evolução do sistema ao longo do tempo.

Embora a descrição detalhada das camadas arquiteturais seja apresentada no Capítulo 4, é importante destacar que a aplicação foi organizada de forma a respeitar a regra de dependência, princípio central da Clean Architecture, segundo o qual as dependências devem sempre apontar para as camadas mais internas (MARTIN, 2017). Por conseguinte, componentes externos, como controllers e repositórios, dependem de abstrações definidas no domínio, e não o contrário.

Adicionalmente, a aplicação incorpora mecanismos complementares que contribuem para sua robustez, como validação de dados de entrada na camada de apresentação, tratamento centralizado de exceções e um controle básico de acesso baseado na identificação do usuário. Esses mecanismos são implementados nas camadas externas da arquitetura, evitando a contaminação da lógica de negócio por detalhes técnicos. Essa organização revela, na prática, como a combinação entre o estilo arquitetural REST e os princípios da Clean Architecture permite a construção de sistemas backend mais organizados, desacoplados e preparados para evolução.

3.5 TECNOLOGIAS E FERRAMENTAS UTILIZADAS

Para o desenvolvimento da aplicação, foi utilizada a plataforma Node.js, que permite a execução de código JavaScript e TypeScript no lado do servidor. O Node.js é amplamente adotado na construção de aplicações backend modernas devido ao seu modelo orientado a eventos e à sua capacidade de lidar com alto volume de requisições de forma eficiente. Sua escolha neste trabalho se justifica pela maturidade do ecossistema e pela ampla adoção em arquiteturas baseadas em serviços.

A linguagem TypeScript foi adotada com o objetivo de adicionar tipagem estática ao JavaScript (BIERMAN; ABADI; TORGERSEN, 2014), proporcionando maior segurança durante o desenvolvimento e facilitando a manutenção do código. A utilização de tipos explícitos contribui para a detecção antecipada de erros e melhora a legibilidade e a organização do sistema, especialmente em aplicações estruturadas em múltiplas camadas, como proposto pela Clean Architecture.

Como framework backend, foi utilizado o NestJS, que oferece suporte nativo a conceitos como injeção de dependência, modularização e organização estrutural do código. Essas características são particularmente relevantes para a aplicação da Clean Architecture, pois facilitam a implementação da inversão de dependência e a separação de responsabilidades entre camadas. O uso do NestJS contribui, portanto, para a organização do sistema sem comprometer o isolamento do domínio.

Para o acesso a dados, foi utilizado o Prisma ORM, responsável pela comunicação com o banco de dados relacional PostgreSQL. A escolha do Prisma se justifica por sua interface tipada e por sua capacidade de integração com TypeScript, o que favorece a consistência do código. No entanto, é importante destacar que o uso do ORM foi restrito à camada de infraestrutura, garantindo que detalhes de persistência não afetem as regras de negócio, em conformidade com os princípios da Clean Architecture.

3.6 ORGANIZAÇÃO DO PROJETO

A organização física do código reflete a separação em camadas da Clean Architecture. O projeto adota uma estrutura de diretórios na qual cada diretório de primeiro nível corresponde a uma das camadas da arquitetura, conforme apresentado na Figura 7.

Figura 7 – Estrutura de diretórios do projeto



Fonte: elaborado pelos autores (2026).

O diretório de domínio concentra as entidades, os objetos de valor e os contratos de repositório, correspondendo ao núcleo de regras de negócio. O diretório de aplicação abriga os casos de uso, além dos objetos de transferência de dados e dos mapeadores responsáveis pela conversão entre entidades e esses objetos. O diretório de apresentação reúne os adaptadores de interface, como controladores, esquemas de validação, pipes, guards e filtros. Por fim, o diretório de infraestrutura concentra os detalhes de framework e de persistência, incluindo o serviço de acesso ao banco de dados, as implementações concretas dos repositórios e os módulos de composição da aplicação.

O código-fonte completo da aplicação, incluindo os arquivos não reproduzidos integralmente neste documento, está disponível no repositório público do projeto, acessível em <https://github.com/rocha-vitor/to-do-list> .

O próximo capítulo apresenta o manual de implementação propriamente dito. A exposição percorre o sistema de dentro para fora, em conformidade com a regra de dependência: inicia pelas entidades e pelos objetos de valor, avança para os casos de uso, em seguida para os adaptadores de interface e, por fim, para a camada de infraestrutura. Em cada camada, o código correspondente é apresentado e analisado, com destaque para as boas práticas adotadas e para os erros que devem ser evitados.

4 MANUAL DE IMPLEMENTAÇÃO DA CLEAN ARCHITECTURE

Neste capítulo é apresentado o manual de implementação da Clean Architecture, elaborado a partir do estudo de caso desenvolvido neste trabalho. A fundamentação conceitual da arquitetura, incluindo o modelo de camadas e a regra de dependência, foi tratada na Seção 2.3 e não é retomada aqui em detalhe. Cabe recordar, no entanto, que a Clean Architecture não constitui uma proposta inteiramente nova, mas a unificação de princípios já presentes em abordagens anteriores de separação em camadas, como a arquitetura hexagonal, também denominada arquitetura de portas e adaptadores (Grebenuk e Pavlenko, 2019), e a arquitetura em cebola (Lano e Tehrani, 2023).

O capítulo está organizado segundo as camadas da arquitetura, da mais interna para a mais externa, acompanhando o sentido da regra de dependência. Inicialmente é tratada a camada de entidades, na qual residem as regras de negócio mais estáveis. Em seguida, são abordadas as camadas de casos de uso, de adaptadores de interface e de infraestrutura. Para cada camada, adota-se a mesma estrutura de apresentação: primeiro descreve-se o procedimento geral de implementação, de modo que possa ser reaplicado a outros projetos, e depois ele é ilustrado com trechos do código desenvolvido. Ao final de cada camada, uma síntese consolida as boas práticas observadas e os equívocos a serem evitados. Ao final, duas seções complementares abordam a travessia de fronteiras entre as camadas e a testabilidade do sistema.

4.1 ENTIDADES

A camada de entidades ocupa a posição mais interna da arquitetura e concentra as regras de negócio mais gerais e estáveis do sistema (Martin, 2017). Tais regras independem de frameworks, de mecanismos de persistência ou de interfaces de usuário, de maneira que sua função primordial é preservar a integridade do domínio diante de mudanças tecnológicas. Nessa perspectiva, as entidades devem encapsular tanto os dados críticos do negócio quanto os comportamentos diretamente associados a eles, fazendo com que as demais camadas dependam do domínio, e não o contrário.

Essa concepção aproxima-se da noção de modelo de domínio rico discutida por Evans (2003), na qual o conhecimento do negócio é expresso em objetos que combinam estado e comportamento, em vez de estruturas voltadas apenas ao transporte de dados. No estudo de caso, a camada de entidades foi responsável por representar os conceitos fundamentais de um

sistema de gerenciamento de listas de tarefas, organizados em torno de duas entidades principais, uma lista de tarefas e uma tarefa, e de um objeto de valor destinado a representar a identidade dos elementos do domínio de forma consistente.

A partir dessa caracterização, a implementação de uma entidade nesta arquitetura pode ser conduzida segundo um procedimento geral, ilustrado nas subseções seguintes. Inicialmente, define-se a estrutura de dados necessária à criação da entidade, distinguindo os atributos obrigatórios dos opcionais. Em seguida, declaram-se os atributos como privados, de forma a impedir alterações diretas em seu estado. Posteriormente, concentra-se no construtor a validação das regras essenciais, garantindo que a entidade nunca exista em estado inválido. Por fim, expõe-se um método de fábrica como ponto único e controlado de criação, evitando o uso direto do construtor.

4.1.1 Entidade TaskList

A primeira entidade implementada representa uma lista de tarefas pertencente a um usuário. No domínio do sistema, uma lista só é considerada válida quando está associada a um usuário e possui um título adequado, regras que, por serem essenciais ao funcionamento da aplicação, foram implementadas na própria entidade, e não em camadas externas.

A estrutura de dados utilizada na criação da entidade é definida por uma interface de propriedades, apresentada na Figura 8. Nela, o identificador é opcional, o que permite que a própria entidade gere sua identidade quando necessário, ao passo que a referência ao usuário é obrigatória, reforçando a regra de que toda lista deve estar associada a um usuário específico.

Figura 8 – Interface TaskListProps

```
src/domain/entities/task-list.entity.ts

4  export interface TaskListProps {
5      id?: UuidV0;
6      userId: UuidV0;
7      title: string;
8      createdAt?: Date;
9      updatedAt?: Date;
10 }
```

Fonte: elaborado pelos autores (2026).

A definição da entidade é apresentada na Figura 9. Seus atributos são declarados como privados, decisão que impede alterações diretas no estado interno e assegura que qualquer

modificação ocorra apenas por meio de métodos controlados. Esse encapsulamento é central para preservar a consistência das regras de negócio.

Figura 9 – Definição da classe TaskList e atributos internos

```
src/domain/entities/task-list.entity.ts

12  export class TaskList {
13      private _id: UuidVO;
14      private _userId: UuidVO;
15      private _title: string;
16      private _createdAt: Date;
17      private _updatedAt: Date;
```

Fonte: elaborado pelos autores (2026).

O construtor, apresentado na Figura 10, é mantido privado, de modo a impedir a criação da entidade sem o cumprimento das regras do domínio. Nele, o título é validado antes de ser atribuído, garantindo que a entidade nunca exista em estado inválido. Caso um identificador não seja fornecido, a própria entidade gera sua identidade, o que reforça sua independência em relação a detalhes externos, como o banco de dados.

Figura 10 – Construtor da entidade TaskList

```
src/domain/entities/task-list.entity.ts

19  private constructor(props: TaskListProps) {
20      this.validateTitle(props.title);
21
22      this._id = props.id ?? UuidVO.generate();
23      this._userId = props.userId;
24      this._title = props.title.trim();
25      this._createdAt = props.createdAt ?? new Date();
26      this._updatedAt = props.updatedAt ?? new Date();
27  }
```

Fonte: elaborado pelos autores (2026).

A criação de instâncias é intermediada por um método de fábrica, apresentado na Figura 11, que oferece um ponto único e controlado de construção. Esse padrão evita o uso direto do construtor e centraliza as regras de criação, assegurando que todas as instâncias sejam produzidas de maneira consistente.

Figura 11 – Método de fábrica create da entidade TaskList

```
src/domain/entities/task-list.entity.ts

29   static create(props: TaskListProps): TaskList {
30     return new TaskList(props);
31   }
```

Fonte: elaborado pelos autores (2026).

4.1.2 Entidade Task

A segunda entidade representa uma tarefa individual associada a uma lista. Além de manter os dados próprios da tarefa, ela controla seu estado de conclusão e assegura a validade de sua descrição, comportamentos que pertencem ao domínio e, por isso, nela foram encapsulados.

A interface de propriedades da tarefa, apresentada na Figura 12, mostra a associação obrigatória com uma lista, sem a qual a tarefa não pode existir.

Figura 12 – Interface TaskProps

```
src/domain/entities/task.entity.ts

4   export interface TaskProps {
5     id?: UuidV0;
6     taskListId: UuidV0;
7     description: string;
8     completed?: boolean;
9     createdAt?: Date;
10    updatedAt?: Date;
11  }
```

Fonte: elaborado pelos autores (2026).

A definição da entidade, na Figura 13, mantém os atributos privados, protegendo o estado interno contra alterações diretas.

Figura 13 – Definição da classe Task e atributos internos

```
src/domain/entities/task.entity.ts

13  export class Task {
14      private _id: UuidV0;
15      private _taskListId: UuidV0;
16      private _description: string;
17      private _completed: boolean;
18      private _createdAt: Date;
19      private _updatedAt: Date;
```

Fonte: elaborado pelos autores (2026).

O construtor privado, apresentado na Figura 14, valida a descrição antes de atribuí-la e, à semelhança da lista de tarefas, gera a identidade quando esta não é fornecida, preservando a consistência da entidade desde sua criação.

Figura 14 – Construtor da entidade Task

```
src/domain/entities/task.entity.ts

21  private constructor(props: TaskProps) {
22      this.validateDescription(props.description);
23
24      this._id = props.id ?? UuidV0.generate();
25      this._taskListId = props.taskListId;
26      this._description = props.description.trim();
27      this._completed = props.completed ?? false;
28      this._createdAt = props.createdAt ?? new Date();
29      this._updatedAt = props.updatedAt ?? new Date();
30  }
```

Fonte: elaborado pelos autores (2026).

A criação é novamente intermediada por um método de fábrica, apresentado na Figura 15, que centraliza as regras de construção e dispensa o uso direto do construtor.

Figura 15 – Método de fábrica create da entidade Task

```
src/domain/entities/task.entity.ts

32  static create(props: TaskProps): Task {
33      return new Task(props);
34  }
```

Fonte: elaborado pelos autores (2026).

A tarefa possui ainda comportamento próprio, ilustrado na Figura 16, responsável por alternar seu estado de conclusão. A existência desse método aponta que a alteração de estado ocorre por intermédio de uma operação de domínio, e não por manipulação externa de atributos.

Figura 16 – Método toggleCompleted

```
src/domain/entities/task.entity.ts

46   updateCompleted(completed: boolean): void {
47       this._completed = completed;
48       this._updatedAt = new Date();
49   }
```

Fonte: elaborado pelos autores (2026).

A validação da descrição, apresentada na Figura 17, concentra na entidade a verificação de que a tarefa possui um texto significativo, impedindo a criação de tarefas vazias ou inconsistentes.

Figura 17 – Validação da descrição da tarefa

```
src/domain/entities/task.entity.ts

59   }
60
61   markAsIncomplete(): void {
62       this._completed = false;
63       this._updatedAt = new Date();
64   }
65
66   private validateDescription(description: string): void {
67       if (!description || typeof description !== 'string') {
68           throw new DomainError('Description is required and must be a string');
69       }
70
71       const trimmedDescription = description.trim();
72       if (trimmedDescription.length === 0) {
```

Fonte: elaborado pelos autores (2026).

Desta forma, a tarefa exemplifica uma entidade que combina dados e comportamento, mantém seu estado protegido e expressa as regras do domínio de maneira autônoma.

4.1.3 Objeto de Valor UuidVO e identidade no domínio

Além das entidades, o domínio recorre a um objeto de valor para representar a identidade de seus elementos. Diferentemente de uma entidade, um objeto de valor não possui identidade própria e é definido apenas por seus atributos, sendo tratado como imutável (Evans,

2003). O emprego desse recurso, em lugar de tipos primitivos como cadeias de caracteres, confere significado de domínio ao identificador e reduz a repetição de validações ao longo do código.

No projeto, esse objeto de valor reúne as responsabilidades de gerar novos identificadores, validar identificadores recebidos, reconstruir identificadores provenientes da persistência, comparar identidades e expor de forma controlada o valor interno.

A Figura 18 mostra a definição do objeto de valor e seu construtor, mantido privado para assegurar que toda instância passe pelas verificações apropriadas.

Figura 18 – Definição do objeto de valor UuidVO

```
src/domain/value-objects/uuid.vo.ts
4   export class UuidVO {
5     private constructor(private readonly value: string) {}
```

Fonte: elaborado pelos autores (2026).

Os métodos responsáveis pela criação são apresentados na Figura 19. Um deles gera um novo identificador, enquanto outro reconstrói um identificador já existente, distinção que separa a criação de uma nova identidade da recuperação de uma identidade persistida.

Figura 19 – Métodos de criação do UuidVO

```
src/domain/value-objects/uuid.vo.ts
7   static generate(): UuidVO {
8     return new UuidVO(uuidv7());
9   }
10
11  static from(value: string): UuidVO {
12    if (!this.isValid(value)) {
13      throw new DomainError('Invalid UUID format');
14    }
```

Fonte: elaborado pelos autores (2026).

Na Figura 20, apresenta-se a rotina de validação, na qual se concentra a regra que define um identificador válido. Centralizar essa verificação em um único ponto evita que a mesma lógica seja replicada nas demais camadas.

Figura 20 – Validação do identificador

```
src/domain/value-objects/uuid.vo.ts

30     private static isValid(value: string): boolean {
31         if (!value || typeof value !== 'string') {
32             return false;
33         }
34         const uuidRegex =
35             /^[0-9a-f]{8}-[0-9a-f]{4}-[1-7][0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$/i;
36         return uuidRegex.test(value);
37     }
```

Fonte: elaborado pelos autores (2026).

Por último, a Figura 21 apresenta os mecanismos de comparação entre identidades e de acesso controlado ao valor interno. A comparação por meio de operação específica, em vez do confronto direto de cadeias de caracteres dispersas pelo código, torna o tratamento da identidade mais seguro e expressivo.

Figura 21 – Comparação e exposição controlada do valor

```
src/domain/value-objects/uuid.vo.ts

16     }
17
18     get valueAsString(): string {
19         return this.value;
20     }
21
22     equals(other: UuidVO): boolean {
23         return this.value === other.value;
24     }
25
26     toString(): string {
27         return this.value;
28     }
```

Fonte: elaborado pelos autores (2026).

Dessa forma, o objeto de valor contribui para a clareza e a robustez do domínio, ao reunir em um único tipo a geração, a validação e a comparação de identidades.

4.1.4 Síntese da camada de entidades

A camada de entidades concentra as regras de negócio mais estáveis do sistema, representadas pelas duas entidades principais e pelo objeto de valor de identidade. Essa organização busca preservar o núcleo do sistema diante de mudanças em tecnologias,

frameworks ou mecanismos de persistência, em consonância com o objetivo central da arquitetura (Martin, 2017).

Ao longo desta camada, observaram-se algumas boas práticas recorrentes. As regras de negócio foram encapsuladas nas próprias entidades e validadas no momento da criação, de forma que nenhuma instância pudesse existir em estado inválido. Os atributos foram mantidos privados, protegendo o estado interno contra alterações diretas, e a criação foi intermediada por métodos de fábrica, que centralizaram as regras de construção. Adotaram-se, ainda, objetos de valor em lugar de tipos primitivos para representar conceitos do domínio, e evitou-se qualquer dependência de frameworks ou bibliotecas externas nessa camada.

Em contrapartida, alguns equívocos devem ser evitados. Entre eles, destacam-se tratar as entidades como meras estruturas de dados, expor publicamente seus atributos, validar regras essenciais fora do domínio, inserir lógica de persistência nas entidades e permitir que camadas externas alterem seu estado sem o uso de métodos apropriados.

Cabe reconhecer que essa abordagem implica certo custo. A definição de objetos de valor, construtores privados e métodos de fábrica aumenta a quantidade de código e de arquivos em relação a uma modelagem baseada apenas em estruturas de dados. É possível que, em sistemas de menor porte, esse custo nem sempre se justifique, ainda que tenda a compensar à medida que as regras de negócio se tornam mais numerosas e a longevidade do sistema se amplia.

Com essa base estabelecida, torna-se possível implementar a camada de casos de uso, responsável por orquestrar as entidades sem violar a regra de dependência, tema da próxima seção.

4.2 CASOS DE USO

A camada de casos de uso situa-se imediatamente acima da camada de entidades e abriga as regras de aplicação, isto é, a lógica específica do sistema que orquestra as entidades para realizar as ações pretendidas (Martin, 2017). Diferentemente das regras de negócio mais gerais, que residem nas entidades, as regras de aplicação descrevem como o sistema utiliza o domínio em cada operação concreta.

Os casos de uso funcionam como uma fronteira entre o domínio e as camadas externas. Eles recebem dados em formato simples, coordenam as entidades e devolvem resultados igualmente simples, sem tomar conhecimento de detalhes como o protocolo HTTP, o banco de

dados ou o framework. Desse modo, preserva-se a regra de dependência, uma vez que a lógica da aplicação depende apenas de abstrações, e não de implementações concretas (Grebenuk e Pavlenko, 2019).

A implementação de um caso de uso nesta arquitetura pode ser conduzida segundo um procedimento geral. Inicialmente, define-se a operação como uma classe com um único ponto de entrada, responsável por um fluxo específico da aplicação. Em seguida, declaram-se como dependências as interfaces necessárias, recebidas de forma externa, em lugar de classes concretas. Posteriormente, concentra-se no método principal a sequência da operação, que valida as regras próprias da aplicação, aciona os comportamentos do domínio e recorre às interfaces para persistir ou consultar dados. Finalmente, retorna-se um objeto simples, adequado ao consumo pelas camadas externas, evitando a exposição direta das entidades (Fowler, 2002).

Esse procedimento é ilustrado a seguir por meio da análise de três casos de uso representativos, escolhidos por exercitarem fluxos distintos: a criação de um recurso, a manipulação do estado de uma entidade e a consulta de dados.

4.2.1 Estrutura geral dos casos de uso no projeto

No projeto, cada caso de uso é implementado como uma classe dedicada a uma ação específica da aplicação, cujo método principal expõe de maneira clara o fluxo executado. As dependências, como os contratos de persistência, são recebidas por interface, o que mantém a camada de aplicação independente de detalhes técnicos.

As regras próprias da aplicação concentram-se nessa camada, enquanto as regras internas de cada entidade permanecem no domínio, e os controllers, situados na borda do sistema, não executam lógica de negócio. Os retornos, por sua vez, são mantidos simples e adequados ao uso externo. Essa disciplina assegura que detalhes de infraestrutura não se infiltrem na lógica da aplicação.

4.2.2 Caso de uso representativo I: CreateTask

O primeiro caso de uso analisado é responsável por criar uma nova tarefa associada a uma lista. Por envolver a criação de um recurso, ele exemplifica como a camada de aplicação

recebe os dados de entrada, aciona o domínio para produzir um objeto válido e o persiste mediante uma interface, sem depender de detalhes externos.

A Figura 22 traz a definição do caso de uso. Observa-se que ele recebe, em seu construtor, apenas a interface de persistência, e não uma implementação concreta, o que evidencia a dependência exclusiva de abstrações.

Figura 22 – Definição do caso de uso CreateTaskUseCase

```
src/application/use-cases/task/create-task.use-case.ts

9   export class CreateTaskUseCase {
10     constructor(
11       private readonly taskRepository: TaskRepository,
12       private readonly taskListRepository: TaskListRepository,
13     ) {}
```

Fonte: elaborado pelos autores (2026).

O início do método de execução é apresentado na Figura 23. Antes de criar a tarefa, o caso de uso verifica a existência da lista à qual ela será associada, aplicando uma regra própria da aplicação, e não do domínio.

Figura 23 – Validação da existência da lista de tarefas

```
src/application/use-cases/task/create-task.use-case.ts

15   async execute(
16     taskListId: string,
17     input: CreateTaskInput,
18     userId: string,
19   ): Promise<TaskOutput> {
20     const taskListIdVO = UuidVO.from(taskListId);
21     const userIdVO = UuidVO.from(userId);
22
23     // Verificar se a task list existe e pertence ao usuário
24     const taskList = await this.taskListRepository.findById(
25       taskListIdVO,
26       userIdVO,
27     );
28
29     if (!taskList) {
30       throw new NotFoundError('TaskList', taskListId);
31     }
```

Fonte: elaborado pelos autores (2026).

A criação da entidade é apresentada na Figura 24. A tarefa é construída por meio do método de fábrica da própria entidade, de maneira que as regras internas, como a validação da descrição, permanecem sob responsabilidade do domínio.

Figura 24 – Criação da entidade Task

```
src/application/use-cases/task/create-task.use-case.ts

33     const task = Task.create({
34         taskListId: taskListIdV0,
35         description: input.description,
36     });
```

Fonte: elaborado pelos autores (2026).

Por fim, a Figura 25 apresenta a persistência da tarefa e o retorno do caso de uso. A persistência é realizada por meio do contrato de repositório, e o resultado é devolvido como um objeto simples, apropriado às camadas externas.

Figura 25 – Persistência e retorno do caso de uso CreateTask

```
src/application/use-cases/task/create-task.use-case.ts

38     const savedTask = await this.taskRepository.save(task);
39
40     return TaskMapper.toOutput(savedTask);
```

Fonte: elaborado pelos autores (2026).

Assim, o caso de uso de criação coordena o fluxo sem assumir responsabilidades que pertencem ao domínio ou à infraestrutura.

4.2.3 Caso de uso representativo II: ToggleTaskCompletion

O segundo caso de uso analisado representa um fluxo distinto, voltado à alteração do estado de uma entidade existente. Em vez de criar um recurso, ele recupera uma tarefa, aciona um comportamento do domínio para alternar seu estado de conclusão e persiste o novo estado.

Na Figura 26, exibe-se a definição do caso de uso, que novamente depende apenas da interface de persistência.

Figura 26 – Definição do caso de uso ToggleTaskCompletionUseCase

```
src/application/use-cases/task/toggle-task-completion.use-case.ts

7     export class ToggleTaskCompletionUseCase {
8         constructor(private readonly taskRepository: TaskRepository) {}
```

Fonte: elaborado pelos autores (2026).

A recuperação da tarefa e a verificação de sua existência são apresentadas na Figura 27. Caso a entidade não seja encontrada, o fluxo é interrompido, o que reflete o tratamento, na camada de aplicação, de uma condição própria da operação.

Figura 27 – Recuperação da entidade Task

```
src/application/use-cases/task/toggle-task-completion.use-case.ts

10  async execute(id: string, userId: string): Promise<TaskOutput> {
11    const idV0 = UuidV0.from(id);
12    const userIdV0 = UuidV0.from(userId);
13
14    const task = await this.taskRepository.findById(idV0, userIdV0);
15
16    if (!task) {
17      throw new NotFoundError('Task', id);
18    }
```

Fonte: elaborado pelos autores (2026).

A Figura 28 apresenta a alteração do estado e a persistência. A mudança não é realizada por manipulação direta de atributos, mas por meio de um comportamento da própria entidade, cabendo ao caso de uso apenas coordenar o fluxo e persistir o resultado.

Figura 28 – Alteração de estado e persistência da tarefa

```
src/application/use-cases/task/toggle-task-completion.use-case.ts

20    task.toggleCompleted();
21
22    const updatedTask = await this.taskRepository.update(task);
23
24    return TaskMapper.toOutput(updatedTask);
```

Fonte: elaborado pelos autores (2026).

Assim, o caso de uso de manipulação de estado reforça o papel da camada de aplicação como coordenadora, preservando no domínio a responsabilidade pelas regras internas.

4.2.4 Caso de uso representativo III: GetTasksByList

O terceiro caso de uso analisado corresponde a uma operação de consulta, escolhida para evidenciar que o padrão arquitetural é mantido mesmo em operações simples. Sua função é recuperar as tarefas associadas a uma lista específica.

A Figura 29 exibe a definição do caso de uso, que, à semelhança dos anteriores, depende exclusivamente de uma interface de consulta.

Figura 29 – Definição do caso de uso GetTasksByListUseCase

```
src/application/use-cases/task/get-tasks-by-list.use-case.ts

8   export class GetTasksByListUseCase {
9     constructor(
10      private readonly taskRepository: TaskRepository,
11      private readonly taskListRepository: TaskListRepository,
12    ) {}
```

Fonte: elaborado pelos autores (2026).

O método de execução é apresentado na Figura 30. A consulta é realizada por meio da interface, e os dados são devolvidos em formato simples, evitando a exposição direta das entidades às camadas externas.

Figura 30 – Execução da listagem de tarefas

```
src/application/use-cases/task/get-tasks-by-list.use-case.ts

14   async execute(taskListId: string, userId: string): Promise<TaskOutput[]> {
15     const taskListIdV0 = UuidV0.from(taskListId);
16     const userIdV0 = UuidV0.from(userId);
17
18     // Verificar se a task list existe e pertence ao usuário
19     const taskList = await this.taskListRepository.findById(
20       taskListIdV0,
21       userIdV0,
22     );
23
24     if (!taskList) {
25       throw new NotFoundError('TaskList', taskListId);
26     }
27
28     const tasks = await this.taskRepository.findByTaskListId(
29       taskListIdV0,
30       userIdV0,
31     );
32
33     return TaskMapper.toOutputArray(tasks);
34   }
```

Fonte: elaborado pelos autores (2026).

Ainda que se trate de uma operação trivial, o caso de uso preserva a separação entre a lógica da aplicação e os detalhes de infraestrutura, mantendo a coerência arquitetural observada nos fluxos mais complexos.

4.2.5 Síntese da camada de casos de uso

A análise dos três casos de uso evidencia que a camada de aplicação atua como coordenadora do sistema, orquestrando as entidades sem assumir suas regras internas e sem depender de detalhes externos. Os fluxos de criação, de manipulação de estado e de consulta, ainda que distintos, seguem o mesmo padrão estrutural.

Ao longo desta camada, observaram-se boas práticas recorrentes. Cada ação da aplicação foi representada por um caso de uso específico, que dependia apenas de interfaces e mantinha centralizadas as regras de aplicação, ao passo que as regras internas permaneciam nas entidades. Os controllers não executavam lógica de negócio, a infraestrutura não decidia fluxos da aplicação e os retornos eram estruturados em formatos simples.

Em contrapartida, alguns equívocos devem ser evitados nessa camada, entre os quais inserir regras de negócio nos controllers, acessar diretamente o mecanismo de persistência dentro dos casos de uso, fazer a camada de aplicação depender de frameworks, transferir comportamento de domínio para os repositórios e retornar objetos de infraestrutura às camadas externas.

Cabe reconhecer que essa organização multiplica o número de classes, já que cada ação dá origem a um caso de uso distinto, o que pode parecer excessivo em sistemas de pequeno porte. Tal custo, no entanto, tende a favorecer a clareza e a evolução do sistema à medida que o número de operações cresce.

Estabelecido o limite entre o domínio e as ações da aplicação, resta examinar como o sistema se comunica com o mundo externo, papel desempenhado pela camada de adaptadores de interface, tratada na próxima seção.

4.3 ADAPTADORES DE INTERFACE

A camada de adaptadores de interface ocupa posição mais externa que as camadas de domínio e de aplicação e tem por função converter dados entre o formato conveniente ao mundo externo e o formato esperado pelo núcleo do sistema (Martin, 2017). É nessa camada que se localizam os elementos responsáveis por traduzir requisições, respostas e contratos, sem que as regras de negócio precisem conhecer tais detalhes.

Dois sentidos de adaptação convivem nessa camada. De um lado, as portas de saída, representadas por interfaces, definem contratos que o núcleo utiliza para alcançar recursos externos, como a persistência, sem depender de suas implementações. De outro, os adaptadores

de entrada, como os controllers, recebem as requisições externas e as encaminham aos casos de uso. Essa organização materializa a noção de portas e adaptadores que precede a Clean Architecture (Grebenuk e Pavlenko, 2019).

De maneira geral, a construção dessa camada envolve definir as interfaces que o núcleo utiliza, prover as implementações concretas por intermédio de um mecanismo de injeção de dependência e concentrar na borda do sistema as preocupações que não pertencem ao domínio, como a recepção de requisições, a validação de entrada, a identificação do usuário e o tratamento de exceções. As subseções seguintes detalham cada um desses elementos no projeto desenvolvido.

4.3.1 Interfaces e portas de saída

As portas de saída do sistema são expressas por interfaces que declaram as operações de que o núcleo necessita, sem especificar como são realizadas. Ao depender dessas abstrações, e não de classes concretas, os casos de uso permanecem isolados dos detalhes de persistência.

A Figura 31 mostra a interface que define o contrato de persistência das tarefas, com as operações exigidas pelo núcleo.

Figura 31 – Interface TaskRepository

```
src/domain/repositories/task.repository.ts

4  export interface TaskRepository {
5      save(task: Task): Promise<Task>;
6      findById(id: UuidV0, userId: UuidV0): Promise<Task | null>;
7      findByTaskListId(taskListId: UuidV0, userId: UuidV0): Promise<Task[]>;
8      update(task: Task): Promise<Task>;
9      delete(id: UuidV0, userId: UuidV0): Promise<void>;
10     deleteByTaskListId(taskListId: UuidV0, userId: UuidV0): Promise<void>;
11 }
```

Fonte: elaborado pelos autores (2026).

De forma análoga, a Figura 32 apresenta a interface correspondente às listas de tarefas. Em ambos os casos, a interface pertence às camadas internas, enquanto sua implementação é relegada à infraestrutura, o que concretiza a inversão de dependência.

Figura 32 – Interface TaskListRepository

```
src/domain/repositories/task-list.repository.ts

4  export interface TaskListRepository {
5      save(taskList: TaskList): Promise<TaskList>;
6      findById(id: UuidV0, userId: UuidV0): Promise<TaskList | null>;
7      findByUserId(userId: UuidV0): Promise<TaskList[]>;
8      update(taskList: TaskList): Promise<TaskList>;
9      delete(id: UuidV0, userId: UuidV0): Promise<void>;
10 }
```

Fonte: elaborado pelos autores (2026).

4.3.2 Tokens de injeção de dependência

Para que o núcleo receba, em tempo de execução, a implementação concreta de cada interface, sem a ela se acoplar, recorre-se a um mecanismo de injeção de dependência (Fowler, 2002). A associação entre a interface e sua implementação é estabelecida por meio de identificadores, comumente denominados tokens.

Na Figura 33, apresenta-se os tokens utilizados para as interfaces de repositório. Esses identificadores permitem que o framework forneça a implementação adequada onde a interface é requerida, mantendo o núcleo alheio à escolha concreta.

Figura 33 – Tokens de repositório

```
src/domain/repositories/task.repository.ts | src/domain/repositories/task-list.repository.ts

src/domain/repositories/task.repository.ts
13  export const TASK_REPOSITORY = Symbol('TaskRepository');

src/domain/repositories/task-list.repository.ts
12  export const TASK_LIST_REPOSITORY = Symbol('TaskListRepository');
```

Fonte: elaborado pelos autores (2026).

4.3.3 Uso das portas nos casos de uso

Definidas as portas e os respectivos tokens, os casos de uso passam a depender exclusivamente das interfaces, recebendo a implementação por injeção.

A Figura 34 ilustra o uso de uma porta de saída em um caso de uso. A dependência é declarada por meio da interface e do token correspondente, de modo que o caso de uso desconhece qual implementação concreta lhe é fornecida. Essa disposição permite substituir o mecanismo de persistência sem alterar a lógica da aplicação.

Figura 34 – Uso da interface TaskRepository em um caso de uso

```

src/application/use-cases/task/toggle-task-completion.use-case.ts

7   export class ToggleTaskCompletionUseCase {
8       constructor(private readonly taskRepository: TaskRepository) {}
9
10      async execute(id: string, userId: string): Promise<TaskOutput> {
11          const idV0 = UuidV0.from(id);
12          const userIdV0 = UuidV0.from(userId);
13
14          const task = await this.taskRepository.findById(idV0, userIdV0);
15
16          if (!task) {
17              throw new NotFoundError('Task', id);
18          }
19
20          task.toggleCompleted();
21
22          const updatedTask = await this.taskRepository.update(task);
23
24          return TaskMapper.toOutput(updatedTask);

```

Fonte: elaborado pelos autores (2026).

4.3.4 Controllers como adaptadores de entrada

No sentido oposto ao das portas de saída, os controllers atuam como adaptadores de entrada, recebendo as requisições externas e traduzindo-as em chamadas aos casos de uso. Sua responsabilidade limita-se à adaptação, sem conter regras de negócio.

A Figura 35 traz a definição do controller responsável pelas tarefas, que recebe, por injeção, os casos de uso de que necessita.

Figura 35 – Definição do controller de tarefas

```

src/presentation/controllers/task.controller.ts

31      DeleteTaskUseCase,
32      } from '../..application';
33
34      @Controller()
35      @UseGuards(UserIdGuard)
36      export class TaskController {
37          constructor(
38              private readonly createTaskUseCase: CreateTaskUseCase,
39              private readonly getTasksByListUseCase: GetTasksByListUseCase,
40              private readonly updateTaskUseCase: UpdateTaskUseCase,

```

Fonte: elaborado pelos autores (2026).

Na Figura 36, exibe-se um de seus métodos, responsável por criar uma tarefa. Observa-se que o controller apenas extrai os dados da requisição, delega a operação ao caso de uso correspondente e devolve a resposta, sem decidir regras da aplicação.

Figura 36 – Método de criação de tarefa no controller

```
src/presentation/controllers/task.controller.ts

49   @Req() req: AuthenticatedRequest,
50   ) {
51     return await this.getTasksByListUseCase.execute(params.listId, req.userId);
52   }
53
54   @Post('task-lists/:listId/tasks')
55   async createTask(
56     @Param(new ZodValidationPipe(taskListIdParamsSchema))
```

Fonte: elaborado pelos autores (2026).

4.3.5 Validação de dados na borda com Pipes

A validação do formato dos dados recebidos constitui uma preocupação de borda, e não uma regra de domínio. Por essa razão, ela é realizada antes que a requisição alcance o caso de uso, evitando que dados malformados penetrem no núcleo.

A Figura 37 apresenta o esquema de validação utilizado na criação de uma tarefa, que descreve a estrutura e as restrições esperadas para os dados de entrada.

Figura 37 – Schema de validação para criação de tarefa

```
src/presentation/schemas/task.schema.ts

3   export const createTaskSchema = z.object({
4     description: z
5       .string()
6       .trim()
7       .min(1, 'Description cannot be empty')
8       .max(1000, 'Description cannot exceed 1000 characters'),
9   });
```

Fonte: elaborado pelos autores (2026).

A Figura 38 exibe o componente que aplica esse esquema às requisições. Caso os dados não estejam conformes, a requisição é rejeitada na borda, de modo que o caso de uso receba apenas dados já validados.

Figura 38 – Pipe de validação com Zod

```

src/presentation/pipes/zod-validation.pipe.ts

5  @Injectable()
6  export class ZodValidationPipe implements PipeTransform {
7      constructor(private schema: ZodSchema) {}
8
9      transform(value: any) {
10         try {
11             const parsedValue = this.schema.parse(value);
12             return parsedValue;
13         } catch (error) {
14             if (error instanceof ZodError) {
15                 const validationErrors = error.issues.map((err) => ({
16                     field: err.path.join('.'),
17                     message: err.message,
18                     code: err.code,
19                 }));
20
21                 throw new BadRequestException({
22                     message: 'Validation failed',
23                     code: 'VALIDATION_ERROR',
24                     details: validationErrors,
25                 });
26             }
27             throw error;
28         }
29     }

```

Fonte: elaborado pelos autores (2026).

4.3.6 Identificação do usuário via Guard (x-user-id)

A identificação do usuário responsável pela requisição é igualmente uma responsabilidade de borda. No sistema, essa identificação é tratada por um componente que intercepta a requisição antes de seu processamento.

A Figura 39 mostra esse componente, responsável por verificar a presença e a validade da identificação do usuário.

Figura 39 – Guard de identificação do usuário

```

src/presentation/guards/user-id.guard.ts

8
9  @Injectable()
10 export class UserIdGuard implements CanActivate {
11     canActivate(context: ExecutionContext): boolean {
12         const request = context.switchToHttp().getRequest<AuthenticatedRequest>();

```

Fonte: elaborado pelos autores (2026).

A Figura 40 detalha a verificação aplicada à identificação recebida. Ao concentrar essa lógica na borda, evita-se que os casos de uso precisem lidar com aspectos de autenticação, mantendo o núcleo voltado apenas às regras de negócio.

Figura 40 – Validação do x-user-id

```
src/presentation/guards/user-id.guard.ts

13     const userId = request.headers['x-user-id'] as string;
14
15     if (!userId || typeof userId !== 'string') {
16         throw new BadRequestException({
17             message: 'Missing or invalid x-user-id header',
18             code: 'MISSING_USER_ID',
19         });
20     }
21
22     // Validate UUID format
23     const uuidRegex =
24         /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i;
25     if (!uuidRegex.test(userId)) {
26         throw new BadRequestException({
27             message: 'Validation failed',
28             code: 'VALIDATION_ERROR',
29             details: [
30                 {
31                     field: 'x-user-id',
32                     message: 'Invalid UUID format',
33                     code: 'invalid_string',
34                 },
35             ],
36         });
37     }
38
39     request.userId = userId;
```

Fonte: elaborado pelos autores (2026).

4.3.7 Tratamento centralizado de exceções

O tratamento de erros constitui outra preocupação transversal que, mantida na borda, evita a dispersão de lógica de resposta pelo núcleo. No sistema, as exceções originadas nas camadas internas são capturadas em um ponto único e convertidas em respostas apropriadas.

Na Figura 41, apresenta-se o componente responsável por capturar as exceções de forma centralizada.

Figura 41 – Filtro centralizado de exceções

```
src/presentation/filters/global-exception.filter.ts

18  }
19
20  @Catch()
21  export class GlobalExceptionHandler implements ExceptionFilter {
22    private readonly logger = new Logger(GlobalExceptionHandler.name);
23
24    catch(exception: unknown, host: ArgumentsHost) {
25      const ctx = host.switchToHttp();
26      const response = ctx.getResponse<Response>();
27
28      let status: number;
29      let ErrorResponse: ErrorResponse;
```

Fonte: elaborado pelos autores (2026).

A Figura 42 traz a conversão das exceções em respostas HTTP. Com isso, o domínio e os casos de uso podem sinalizar condições de erro mediante exceções próprias, sem conhecer o formato das respostas externas, cabendo à borda a tradução correspondente.

Figura 42 – Conversão de exceções em respostas HTTP

```

src/presentation/filters/global-exception.filter.ts

30
31     if (exception instanceof HttpException) {
32         status = exception.getStatus();
33         const exceptionResponse = exception.getResponse();
34
35         if (typeof exceptionResponse === 'object' && exceptionResponse !== null) {
36             errorResponse = exceptionResponse as ErrorResponse;
37         } else {
38             errorResponse = {
39                 message: exceptionResponse.toString(),
40                 code: 'HTTP_ERROR',
41             };
42         }
43     } else if (exception instanceof NotFoundError) {
44         status = HttpStatus.NOT_FOUND;
45         errorResponse = {
46             message: exception.message,
47             code: 'RESOURCE_NOT_FOUND',
48         };
49     } else if (exception instanceof DomainError) {
50         status = HttpStatus.BAD_REQUEST;
51         errorResponse = {
52             message: exception.message,
53             code: 'DOMAIN_ERROR',
54         };
55     } else if (exception instanceof ApplicationError) {
56         status = HttpStatus.BAD_REQUEST;
57         errorResponse = {
58             message: exception.message,
59             code: 'APPLICATION_ERROR',
60         };
61     } else if (exception instanceof Error) {
62         status = HttpStatus.INTERNAL_SERVER_ERROR;
63         errorResponse = {
64             message: 'Internal server error',
65             code: 'INTERNAL_ERROR',
66         };
67
68         this.logger.error('Unexpected error:', exception.stack);
69     } else {
70         status = HttpStatus.INTERNAL_SERVER_ERROR;
71         errorResponse = {
72             message: 'Internal server error',
73             code: 'INTERNAL_ERROR',
74         };
75
76         this.logger.error('Unknown exception:', exception);
77     }
78
79     response.status(status).json(errorResponse);

```

Fonte: elaborado pelos autores (2026).

4.3.8 Síntese da camada de adaptadores de interface

A camada de adaptadores de interface, conforme apresentada, cumpre o papel de mediar a comunicação entre o núcleo do sistema e o mundo externo, em ambos os sentidos. As portas de saída, expressas por interfaces e providas por injeção de dependência, mantêm o núcleo independente da persistência, enquanto os adaptadores de entrada concentram, na borda, as preocupações de recepção de requisições, validação, identificação do usuário e tratamento de exceções.

Entre as boas práticas observadas nesta camada, destacam-se a definição de contratos por interface, a delegação das implementações concretas à infraestrutura por meio de injeção de dependência, a restrição dos controllers ao papel de adaptação e a manutenção, na borda, das preocupações que não pertencem ao domínio. Os casos de uso, por sua vez, permaneceram dependentes apenas de abstrações.

Em contrapartida, devem ser evitados equívocos como inserir regras de negócio nos controllers, validar o formato dos dados dentro do núcleo, espalhar o tratamento de erros pelas diversas camadas e fazer os casos de uso dependerem de componentes concretos de infraestrutura.

Definidos os contratos e os adaptadores, resta apresentar a camada que fornece as implementações concretas desses contratos, responsável pelo acesso efetivo a recursos externos, tema da próxima seção.

4.4 INFRAESTRUTURA

A camada de infraestrutura ocupa a posição mais externa da arquitetura e reúne as implementações concretas dos contratos definidos pelas camadas internas, além dos componentes que interagem diretamente com tecnologias externas, como o banco de dados (Martin, 2017). É nesta camada que residem as dependências de frameworks e bibliotecas, mantidas afastadas do núcleo do sistema.

Por situar-se na borda da regra de dependência, a infraestrutura conhece as camadas internas, mas não é por elas conhecida. Suas implementações são fornecidas ao núcleo por injeção de dependência, de forma que uma eventual substituição de tecnologia afete apenas esta camada, preservando o domínio e a aplicação.

De maneira geral, a construção dessa camada envolve prover um ponto de acesso ao mecanismo de persistência, implementar os contratos de repositório definidos pelo núcleo e compor, por intermédio de módulos, a associação entre as interfaces e suas implementações concretas. As subseções seguintes detalham esses elementos no projeto.

4.4.1 PrismaService e acesso ao banco de dados

O acesso ao banco de dados é centralizado em um serviço dedicado, responsável por gerenciar a conexão e por disponibilizar as operações de persistência às demais classes da infraestrutura.

Na Figura 43, exibe-se esse serviço de integração com o mecanismo de persistência. Concentrar a conexão em um único ponto evita a dispersão de detalhes de acesso ao banco de dados pelas diversas classes que dele dependem.

Figura 43 – Serviço de integração com Prisma

```
src/infrastructure/database/prisma.service.ts

4  @Injectable()
5  export class PrismaService
6    extends PrismaClient
7    implements OnModuleInit, OnModuleDestroy
8  {
9    constructor() {
10      super({
11        datasources: {
12          db: {
13            url: process.env.DATABASE_URL,
14          },
15        },
16      });
17    }
18
19    async onModuleInit() {
20      await this.$connect();
21    }
22
23    async onModuleDestroy() {
24      await this.$disconnect();
25    }
26  }
```

Fonte: elaborado pelos autores (2026).

4.4.2 Repositório concreto de tarefas

As implementações concretas dos contratos de repositório constituem o elemento central desta camada. Elas realizam, com o auxílio do serviço de persistência, as operações declaradas pelas interfaces do núcleo, traduzindo entre as entidades do domínio e o formato armazenado no banco de dados. Essa estrutura corresponde ao padrão Repository descrito por Fowler (2002), que intermedia o acesso entre o domínio e a fonte de dados por meio de uma interface semelhante à de uma coleção.

A Figura 44 apresenta a implementação concreta do repositório de tarefas, que recebe o serviço de persistência por injeção e implementa a interface definida no núcleo.

Figura 44 – Repositório concreto de tarefas com Prisma

```
src/infrastructure/repositories/prisma-task.repository.ts

7  @Injectable()
8  export class PrismaTaskRepository implements TaskRepository {
9    constructor(private readonly prisma: PrismaService) {}
10 }
```

Fonte: elaborado pelos autores (2026).

A Figura 45 exibe a operação de persistência de uma tarefa. Os dados da entidade são convertidos para o formato esperado pelo banco de dados antes de serem gravados.

Figura 45 – Persistência da entidade Task

```
src/infrastructure/repositories/prisma-task.repository.ts

11  async save(task: Task): Promise<Task> {
12      const data = {
13          id: task.id.valueAsString,
14          taskListId: task.taskListId.valueAsString,
15          description: task.description,
16          completed: task.completed,
17          createdAt: task.createdAt,
18          updatedAt: task.updatedAt,
19      };
20
21      const saved = await this.prisma.task.create({ data });
22
23      return Task.restore({
24          id: UuidV0.from(saved.id),
25          taskListId: UuidV0.from(saved.taskListId),
26          description: saved.description,
27          completed: saved.completed,
28          createdAt: saved.createdAt,
29          updatedAt: saved.updatedAt,
30      });
31  }
```

Fonte: elaborado pelos autores (2026).

A Figura 46 mostra a operação de recuperação, na qual o registro obtido do banco de dados é restaurado na forma de uma entidade do domínio. Essa conversão, em ambos os sentidos, assegura que o restante do sistema lide apenas com entidades, e não com estruturas próprias do banco de dados.

Figura 46 – Recuperação e restauração da entidade Task

```

src/infrastructure/repositories/prisma-task.repository.ts

33  async findById(id: UuidV0, userId: UuidV0): Promise<Task | null> {
34      const found = await this.prisma.task.findFirst({
35          where: {
36              id: id.valueAsString,
37              taskList: {
38                  userId: userId.valueAsString,
39              },
40          },
41      });
42
43      if (!found) {
44          return null;
45      }
46
47      return Task.restore({
48          id: UuidV0.from(found.id),
49          taskListId: UuidV0.from(found.taskListId),
50          description: found.description,
51          completed: found.completed,
52          createdAt: found.createdAt,
53          updatedAt: found.updatedAt,
54      });
55  }

```

Fonte: elaborado pelos autores (2026).

4.4.3 Repositório concreto de listas de tarefas

De forma análoga, o contrato de persistência das listas de tarefas é implementado por um repositório concreto próprio, que segue a mesma estrutura adotada para as tarefas.

Na Figura 47, apresenta-se a implementação concreta do repositório de listas de tarefas.

Figura 47 – Repositório concreto de listas de tarefas

```

src/infrastructure/repositories/prisma-task-list.repository.ts

7  @Injectable()
8  export class PrismaTaskListRepository implements TaskListRepository {
9      constructor(private readonly prisma: PrismaService) {}

```

Fonte: elaborado pelos autores (2026).

A Figura 48 traz a operação de persistência de uma lista.

Figura 48 – Persistência da entidade TaskList

```
src/infrastructure/repositories/prisma-task-list.repository.ts

11  async save(taskList: TaskList): Promise<TaskList> {
12    const data = {
13      id: taskList.id.valueAsString,
14      userId: taskList.userId.valueAsString,
15      title: taskList.title,
16      createdAt: taskList.createdAt,
17      updatedAt: taskList.updatedAt,
18    };
19
20    const saved = await this.prisma.taskList.create({ data });
21
22    return TaskList.restore({
23      id: UuidV0.from(saved.id),
24      userId: UuidV0.from(saved.userId),
25      title: saved.title,
26      createdAt: saved.createdAt,
27      updatedAt: saved.updatedAt,
28    });
29  }
```

Fonte: elaborado pelos autores (2026).

Na Figura 49, exibe-se a listagem das listas pertencentes a um usuário. Mantém-se, também aqui, a conversão entre as entidades do domínio e os registros do banco de dados, de maneira que a regra de dependência seja preservada.

Figura 49 – Listagem de listas por usuário

```
src/infrastructure/repositories/prisma-task-list.repository.ts

52  async findByUserId(userId: UuidV0): Promise<TaskList[]> {
53    const taskLists = await this.prisma.taskList.findMany({
54      where: {
55        userId: userId.valueAsString,
56      },
57      orderBy: {
58        createdAt: 'asc',
59      },
60    });
61
62    return taskLists.map((taskList) =>
63      TaskList.restore({
64        id: UuidV0.from(taskList.id),
65        userId: UuidV0.from(taskList.userId),
66        title: taskList.title,
67        createdAt: taskList.createdAt,
68        updatedAt: taskList.updatedAt,
69      }),
70    );
71  }
```

Fonte: elaborado pelos autores (2026).

4.4.4 Módulo de repositórios e composição dos contratos

Definidas as implementações concretas, é necessário associá-las às respectivas interfaces, de modo que o núcleo as receba por injeção sem conhecê-las diretamente. Essa associação é estabelecida mediante um módulo.

A Figura 50 apresenta o módulo responsável por relacionar cada contrato de repositório à sua implementação concreta. É nesse ponto que a escolha da tecnologia de persistência se materializa, sem que o núcleo a perceba, o que revela, na prática, a inversão de dependência.

Figura 50 – Módulo de repositórios

```
src/infrastructure/modules/repositories.module.ts

1  import { Module } from '@nestjs/common';
2  import { PrismaTaskListRepository } from '../repositories/prisma-task-list.repository';
3  import { PrismaTaskRepository } from '../repositories/prisma-task.repository';
4  import { TASK_LIST_REPOSITORY, TASK_REPOSITORY } from '../../domain';
5
6  @Module({
7    providers: [
8      {
9        provide: TASK_LIST_REPOSITORY,
10       useClass: PrismaTaskListRepository,
11     },
12     {
13       provide: TASK_REPOSITORY,
14       useClass: PrismaTaskRepository,
15     },
16   ],
17   exports: [TASK_LIST_REPOSITORY, TASK_REPOSITORY],
18 })
```

Fonte: elaborado pelos autores (2026).

4.4.5 Módulo de casos de uso e disponibilização da aplicação

Além dos repositórios, a aplicação precisa disponibilizar seus casos de uso e fornecer-lhes as dependências de que necessitam. Essa composição também é realizada por meio de um módulo.

A Figura 51 exibe o módulo que disponibiliza os casos de uso, associando-os aos contratos de repositório. Por conseguinte, a montagem das dependências do sistema fica concentrada nos módulos, que atuam como ponto de composição da aplicação.

Figura 51 – Módulo de casos de uso

```

src/infrastructure/modules/use-cases.module.ts

21  @Module({
22    imports: [RepositoriesModule],
23    providers: [
24      {
25        provide: CreateTaskListUseCase,
26        useFactory: (taskListRepository: TaskListRepository) =>
27          new CreateTaskListUseCase(taskListRepository),
28        inject: [TASK_LIST_REPOSITORY],
29      },
30      {
31        provide: GetTaskListsUseCase,
32        useFactory: (taskListRepository: TaskListRepository) =>
33          new GetTaskListsUseCase(taskListRepository),
34        inject: [TASK_LIST_REPOSITORY],
35      },
36      {
37        provide: UpdateTaskListUseCase,
38        useFactory: (taskListRepository: TaskListRepository) =>
39          new UpdateTaskListUseCase(taskListRepository),
40        inject: [TASK_LIST_REPOSITORY],
41      },
42      {
43        provide: DeleteTaskListUseCase,
44        useFactory: (
45          taskListRepository: TaskListRepository,
46          taskRepository: TaskRepository,
47        ) => new DeleteTaskListUseCase(taskListRepository, taskRepository),
48        inject: [TASK_LIST_REPOSITORY, TASK_REPOSITORY],
49      },
50      {
51        provide: CreateTaskUseCase,
52        useFactory: (
53          taskRepository: TaskRepository,
54          taskListRepository: TaskListRepository,
55        ) => new CreateTaskUseCase(taskRepository, taskListRepository),
56        inject: [TASK_REPOSITORY, TASK_LIST_REPOSITORY],
57      },
58      {
59        provide: GetTasksByListUseCase,
60        useFactory: (
61          taskRepository: TaskRepository,
62          taskListRepository: TaskListRepository,
63        ) => new GetTasksByListUseCase(taskRepository, taskListRepository),
64        inject: [TASK_REPOSITORY, TASK_LIST_REPOSITORY],
65      },
66      {
67        provide: UpdateTaskUseCase,
68        useFactory: (taskRepository: TaskRepository) =>
69          new UpdateTaskUseCase(taskRepository),
70        inject: [TASK_REPOSITORY],
71      },
72      {
73        provide: ToggleTaskCompletionUseCase,
74        useFactory: (taskRepository: TaskRepository) =>
75          new ToggleTaskCompletionUseCase(taskRepository),
76        inject: [TASK_REPOSITORY],
77      },
78      {
79        provide: DeleteTaskUseCase,
80        useFactory: (taskRepository: TaskRepository) =>
81          new DeleteTaskUseCase(taskRepository),
82        inject: [TASK_REPOSITORY],
83      },
84    ],
85    exports: [
86      CreateTaskListUseCase,
87      GetTaskListsUseCase,
88      UpdateTaskListUseCase,
89      DeleteTaskListUseCase,
90      CreateTaskUseCase,
91      GetTasksByListUseCase,
92      UpdateTaskUseCase,
93      ToggleTaskCompletionUseCase,
94      DeleteTaskUseCase,
95    ],
96  })
97  export class UseCasesModule {}

```

Fonte: elaborado pelos autores (2026).

4.4.6 Síntese da camada de infraestrutura

A camada de infraestrutura, conforme apresentada, fornece as implementações concretas dos contratos definidos pelo núcleo e concentra as dependências de tecnologias

externas. Os repositórios concretos realizam a persistência e a conversão entre entidades e registros, enquanto os módulos compõem as dependências do sistema, associando cada interface à respectiva implementação.

Entre as boas práticas observadas nesta camada, destacam-se a centralização do acesso ao banco de dados, a implementação dos contratos definidos pelo núcleo sem alterá-los, a conversão explícita entre entidades do domínio e registros de persistência e a concentração, em módulos, da associação entre interfaces e implementações. Dessa maneira, as dependências de framework permaneceram restritas a esta camada.

Em contrapartida, devem ser evitados equívocos como expor estruturas próprias do banco de dados às camadas internas, inserir regras de negócio nos repositórios, espalhar a configuração de acesso ao banco de dados por diversas classes e fazer o núcleo depender diretamente das implementações concretas.

Cabe reconhecer que a conversão explícita entre entidades e registros, bem como a definição de interfaces e módulos, acrescenta uma camada de indireção e certo volume de código que poderia ser dispensado em uma solução que acessasse o banco de dados diretamente. Esse custo, contudo, é o que assegura a substituíbilidade da tecnologia e o isolamento do núcleo.

Apresentadas as quatro camadas e a forma como se articulam, restam dois aspectos transversais que complementam o manual. As seções seguintes tratam, respectivamente, da travessia de fronteiras entre as camadas, por intermédio de objetos de transferência e mapeadores, e da testabilidade proporcionada pela arquitetura.

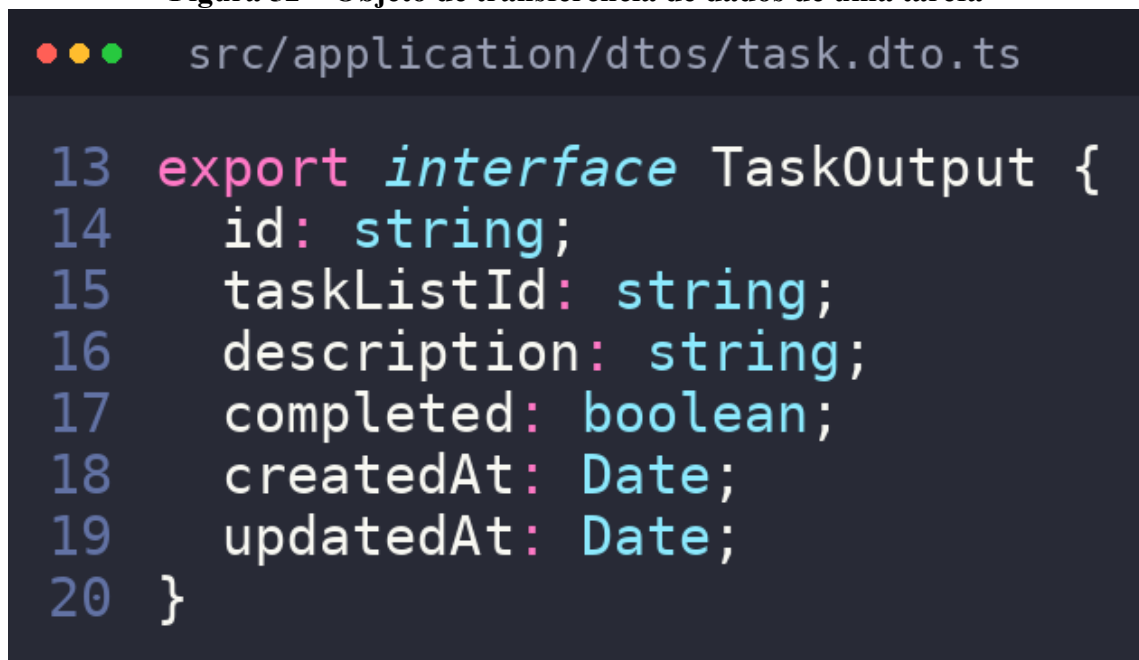
4.5 TRAVESSIA DE FRONTEIRAS COM OBJETOS DE TRANSFERÊNCIA E MAPEADORES

Uma questão recorrente na Clean Architecture diz respeito ao formato dos dados que atravessam as fronteiras entre as camadas. Conforme observado nas seções anteriores, os casos de uso não devem expor diretamente as entidades do domínio às camadas externas, sob pena de acoplar a apresentação à estrutura interna do domínio. Para evitar esse acoplamento, recorre-se a objetos de transferência de dados, conhecidos como DTOs, e a mapeadores responsáveis por converter entre as entidades e esses objetos (Fowler, 2002).

Um objeto de transferência é uma estrutura simples, composta apenas por dados, sem comportamento, destinada a transportar informações entre as camadas. A Figura 52 mostra o

objeto de transferência utilizado para representar uma tarefa nas respostas do sistema. Observa-se que ele contém apenas tipos primitivos, em lugar dos objetos de valor empregados internamente pela entidade.

Figura 52 – Objeto de transferência de dados de uma tarefa



```
src/application/dtos/task.dto.ts

13 export interface TaskOutput {
14     id: string;
15     taskListId: string;
16     description: string;
17     completed: boolean;
18     createdAt: Date;
19     updatedAt: Date;
20 }
```

Fonte: elaborado pelos autores (2026).

A conversão entre a entidade e o objeto de transferência é responsabilidade de um mapeador. Na Figura 53, apresenta-se o mapeador correspondente às tarefas, que recebe uma entidade e produz o objeto de transferência equivalente, extraindo os valores internos, como o identificador, em sua forma textual. Dessa maneira, a entidade permanece encapsulada, e as camadas externas recebem apenas os dados de que necessitam.

Figura 53 – Mapeador da entidade Task para objeto de transferência

```
src/application/mappers/task.mapper.ts

4 export class TaskMapper {
5     static toOutput(task: Task): TaskOutput {
6         return {
7             id: task.id.valueAsString,
8             taskListId: task.taskListId.valueAsString,
9             description: task.description,
10            completed: task.completed,
11            createdAt: task.createdAt,
12            updatedAt: task.updatedAt,
13        };
14    }
15 }
```

Fonte: elaborado pelos autores (2026).

Vale ressaltar que os mapeadores situam-se na camada de aplicação, junto aos casos de uso que os utilizam, e dependem apenas das entidades e dos objetos de transferência, sem qualquer vínculo com detalhes de infraestrutura. Desta forma, a travessia de fronteiras preserva o isolamento do domínio, ao mesmo tempo em que oferece às camadas externas uma representação estável e desacoplada dos dados.

4.6 TESTABILIDADE NA PRÁTICA

Um dos benefícios mais frequentemente atribuídos à Clean Architecture é a testabilidade do núcleo da aplicação (Grebenuk e Pavlenko, 2019). Como os casos de uso dependem apenas de abstrações, e não de implementações concretas de infraestrutura, é possível exercitá-los de forma isolada, substituindo as dependências reais por versões simuladas. Esta seção ilustra, na prática, como essa propriedade se manifesta no projeto.

Para testar um caso de uso sem recorrer ao banco de dados, é suficiente fornecer uma implementação alternativa do contrato de repositório que mantenha os dados em memória. A Figura 54 traz uma implementação dessa natureza, que armazena as tarefas em uma estrutura interna e satisfaz a mesma interface utilizada pelo caso de uso. Por respeitar o contrato definido pelo núcleo, essa implementação pode substituir a versão concreta baseada em banco de dados sem qualquer alteração no caso de uso.

Figura 54 – Repositório de tarefas em memória para testes

```

test/in-memory-task.repository.ts

1 export class InMemoryTaskRepository implements TaskRepository {
2   private tasks: Task[] = [];
3
4   async save(task: Task): Promise<Task> {
5     this.tasks.push(task);
6     return task;
7   }
8
9   async findById(id: UuidV0, userId: UuidV0): Promise<Task | null> {
10    return this.tasks.find((task) => task.id.equals(id)) ?? null;
11  }
12
13  async update(updated: Task): Promise<Task> {
14    const index = this.tasks.findIndex((task) => task.id.equals(updated.id));
15    this.tasks[index] = updated;
16    return updated;
17  }
18
19  // demais operacoes da interface seguem a mesma logica em memoria
20 }

```

Fonte: elaborado pelos autores (2026).

Com essa implementação em memória, torna-se possível verificar o comportamento de um caso de uso de maneira automatizada. Na Figura 55, exibe-se um teste do caso de uso responsável por alternar o estado de conclusão de uma tarefa. O teste cria uma tarefa, fornece-a ao repositório em memória, executa o caso de uso e verifica se o estado de conclusão foi de fato alternado, sem qualquer dependência de banco de dados, framework ou protocolo HTTP.

Figura 55 – Teste de unidade do caso de uso de alternância de conclusão

```

test/toggle-task-completion.use-case.spec.ts

1 describe('ToggleTaskCompletionUseCase', () => {
2   it('alterna o estado de conclusao de uma tarefa', async () => {
3     const repository = new InMemoryTaskRepository();
4     const useCase = new ToggleTaskCompletionUseCase(repository);
5
6     const listId = UuidV0.generate();
7     const task = Task.create({ taskListId: listId, description: 'Estudar' });
8     await repository.save(task);
9
10    const output = await useCase.execute(
11      task.id.valueAsString,
12      UuidV0.generate().valueAsString,
13    );
14
15    expect(output.completed).toBe(true);
16  });
17 });

```

Fonte: elaborado pelos autores (2026).

Embora a suíte de testes do sistema não constitua o foco deste trabalho, o exemplo mostra que a arquitetura adotada favorece a verificação isolada das regras de aplicação. É possível que esse ganho seja particularmente relevante em sistemas de maior porte, nos quais a

dependência direta de infraestrutura tende a dificultar a escrita de testes. Os resultados decorrentes dessa e das demais propriedades observadas são discutidos no capítulo seguinte.

5 RESULTADOS E ANÁLISE

Neste capítulo são discutidos os resultados observados com a adoção da Clean Architecture no desenvolvimento do sistema de gerenciamento de listas de tarefas. A análise apoia-se em evidências extraídas do próprio código, em indicadores estruturais e na comparação com achados da literatura, e procura, ao final, reconhecer de forma sistemática as limitações do estudo. Cada propriedade discutida nas seções seguintes é primeiro caracterizada, em seguida confrontada com a evidência observada e, quando pertinente, situada em relação a trabalhos correlatos. Cabe ressaltar que se trata de um estudo de caso de natureza qualitativa, de modo que os resultados aqui apresentados devem ser interpretados como indícios, e não como demonstrações estatisticamente generalizáveis.

5.1 INDEPENDÊNCIA E SUBSTITUIBILIDADE DO NÚCLEO

Um dos principais resultados observados foi a independência do núcleo do sistema em relação a frameworks e tecnologias externas. A verificação do código revela que as camadas de domínio e de aplicação, que reúnem as entidades, os objetos de valor e os casos de uso, não contêm qualquer dependência direta de NestJS, de Prisma ou do protocolo HTTP. Das 41 unidades de código que compõem a aplicação, as 23 pertencentes a essas duas camadas permaneceram livres de tais dependências, ao passo que o código acoplado a tecnologias específicas concentrou-se nas camadas externas de adaptadores e de infraestrutura. Essa distribuição materializa, no nível do código, a regra de dependência que orienta a abordagem, segundo a qual as dependências devem apontar sempre das camadas externas para as internas, nunca o contrário.

A independência observada tem implicações práticas relevantes. Como as regras de negócio não fazem referência a nenhum mecanismo de entrega ou de persistência, a lógica central poderia, em princípio, ser reaproveitada em um contexto de execução distinto, como uma aplicação de linha de comando ou um serviço acionado por mensagens, sem reescrita das entidades e dos casos de uso. Essa propriedade aproxima-se do que relata Brown et al. (2021), que, ao refatorar um sistema preexistente para a Clean Architecture, observaram que o isolamento do domínio facilitou a substituição de componentes externos. De modo semelhante, Kaczmariski (2024) descreve, na construção de uma API com a mesma abordagem, que a

organização em camadas permitiu tratar a tecnologia de comunicação como um detalhe periférico ao núcleo.

Diretamente relacionada a essa independência está a substituíbilidade dos detalhes técnicos. Conforme discutido no capítulo anterior, os casos de uso dependem apenas de interfaces de repositório, definidas no próprio núcleo, enquanto as implementações concretas, baseadas em Prisma, residem na camada de infraestrutura. Em decorrência disso, a substituição do mecanismo de persistência, por exemplo a troca do banco de dados relacional por outro sistema de armazenamento, implicaria fornecer uma nova implementação do contrato já existente, sem alteração das regras de negócio que dele se utilizam. Raciocínio análogo aplica-se ao framework de entrada, que, por não ser referenciado pelo núcleo, poderia ser substituído com impacto restrito às camadas externas. Os resultados sugerem, portanto, que a inversão de dependência, princípio central da abordagem (Martin, 2017), foi efetivamente concretizada no projeto, e não apenas enunciada.

5.2 MANUTENIBILIDADE E CLAREZA ESTRUTURAL

A organização em camadas também se refletiu na clareza estrutural do código. A separação entre entidades, casos de uso, adaptadores de interface e infraestrutura tornou explícito o papel de cada componente e distribuiu as 41 unidades de código de forma coerente com suas responsabilidades, sendo oito na camada de domínio, quinze na de aplicação, dez na de adaptadores e oito na de infraestrutura. Essa distribuição aponta que a maior parte do código se concentra nas camadas que expressam as regras de negócio e a orquestração dos casos de uso, e não em detalhes técnicos, o que é coerente com a intenção de manter o domínio como elemento central do sistema.

Um indício adicional de clareza estrutural está na correspondência direta entre as funcionalidades expostas e os elementos internos. As nove operações disponibilizadas pela interface da aplicação correspondem a nove casos de uso distintos, cada qual encapsulando uma única ação do sistema. Essa relação de um para um tende a reduzir o esforço de localização ao longo da manutenção, uma vez que a alteração de um comportamento específico remete a um componente bem delimitado, em vez de exigir a inspeção de módulos extensos e multifuncionais. A predominância de classes pequenas e de responsabilidade única favorece, ainda, a leitura do código por desenvolvedores que não participaram de sua escrita.

Tais observações mostram-se coerentes com achados relatados na literatura. Estudos que investigaram a aplicação de arquiteturas em camadas e da Clean Architecture, como os de Nugroho, Kusumo e Alibasa (2022) e de Beltrão, Farzat e Travassos (2020), associam essa forma de organização a ganhos de manutenibilidade, o segundo inclusive por meio de métricas obtidas com ferramentas de análise estática integradas ao ciclo de desenvolvimento. A norma ISO/IEC 25010 oferece o vocabulário que confere precisão a essa noção, ao desdobrar a manutenibilidade em subcaracterísticas como modularidade, analisabilidade e modificabilidade (ISO, 2023), atributos para os quais a separação observada no projeto tende a contribuir. Convém ressaltar, no entanto, que tais ganhos não foram medidos quantitativamente no presente trabalho, de forma que a avaliação aqui realizada permanece de caráter estrutural e qualitativo.

5.3 TESTABILIDADE

A testabilidade do sistema constitui outro resultado decorrente da arquitetura adotada (Grebenuk e Pavlenko, 2019). Conforme demonstrado na Seção 4.6, um caso de uso pôde ser exercitado de forma isolada, mediante a substituição do repositório concreto por uma implementação em memória que satisfaz o mesmo contrato, sem qualquer dependência de banco de dados, framework ou protocolo HTTP. O mecanismo que viabiliza essa substituição é a própria inversão de dependência, pois, como o caso de uso recebe o repositório mediante uma interface, e não instancia diretamente a implementação concreta, torna-se possível fornecer-lhe um substituto controlado durante o teste.

A relevância dessa propriedade torna-se mais clara quando contrastada com o cenário oposto. Em sistemas nos quais as regras de negócio dependem diretamente de frameworks e de mecanismos de persistência, a verificação de um comportamento isolado costuma exigir a preparação de um banco de dados, a inicialização do framework ou a simulação de requisições, o que torna os testes mais lentos, mais frágeis e mais custosos de manter. Ao concentrar essas dependências nas camadas externas, a arquitetura adotada permite que as regras de aplicação sejam verificadas por meio de testes simples e de execução rápida. Ainda assim, reconhece-se que a construção de uma suíte de testes abrangente, capaz de cobrir todas as camadas e cenários, não integrou o escopo deste trabalho, tendo o exemplo apresentado caráter ilustrativo.

5.4 ISOLAMENTO DAS PREOCUPAÇÕES DE BORDA

A decisão de tratar a validação de entrada, a identificação do usuário e o tratamento de exceções como responsabilidades da camada de adaptadores mostrou-se adequada à preservação do núcleo. No projeto, essas preocupações foram concretizadas, respectivamente, por um mecanismo de validação de esquema na fronteira de entrada, por um componente de identificação do usuário a partir do cabeçalho da requisição e por um filtro centralizado de conversão de exceções em respostas apropriadas. Ao concentrar essas funções na borda do sistema, evitou-se que detalhes de protocolo e de transporte penetrassem no domínio e nos casos de uso, que permaneceram voltados exclusivamente às regras de negócio.

Esse arranjo guarda relação com a observação de Aniche, Yoder e Kon (2019) de que parte do acoplamento em sistemas reais é induzida pelas próprias tecnologias adotadas, que pressionam o código a incorporar suas convenções. Ao confinar as preocupações de borda a adaptadores específicos, a arquitetura delimita o alcance dessa pressão e reduz o risco de que ela contamine o núcleo. Os resultados indicam que essa separação contribuiu para um domínio mais coeso, embora se deva reconhecer que ela introduz um custo, na forma de um número maior de componentes e de um fluxo de dados que atravessa mais etapas, cujo efeito sobre a manutenção a longo prazo não foi objeto de medição neste trabalho.

5.5 ATENDIMENTO AOS OBJETIVOS DA PESQUISA

Os resultados discutidos permitem retomar, de forma explícita, os objetivos enunciados na introdução. O primeiro objetivo específico, de compreender os conceitos fundamentais de arquitetura de software e da Clean Architecture, foi contemplado na fundamentação apresentada no Capítulo 2. O segundo, de estruturar a aplicação em camadas bem definidas respeitando a regra de dependência, e o terceiro, de implementar entidades, casos de uso, interfaces e adaptadores, concretizaram-se na construção descrita no Capítulo 4. O quarto objetivo, de demonstrar por intermédio de código a separação entre domínio e infraestrutura, foi atendido pela implementação prática apresentada no Capítulo 4 e discutido, sob uma perspectiva estrutural, a partir dos resultados apresentados na Seção 5.1. O quinto objetivo, de sistematizar boas práticas e equívocos em formato de manual, materializou-se nas sínteses ao final de cada camada, que reúnem as recomendações e as armadilhas observadas durante a implementação. O sexto, de analisar os benefícios em termos de organização e manutenibilidade, constitui o objeto central deste capítulo e foi desenvolvido por meio de uma discussão qualitativa dos aspectos observados na implementação, considerando também as limitações do estudo. Desta

maneira, considera-se que os objetivos específicos foram contemplados, em consonância com o objetivo geral de propor um manual de implementação da Clean Architecture em projetos Node.js com TypeScript e discutir as implicações da aplicação de seus princípios para a organização, manutenibilidade e evolução do software.

O quinto objetivo, de sistematizar boas práticas e equívocos em formato de manual, materializou-se nas sínteses ao final de cada camada, que reúnem as recomendações e as armadilhas observadas durante a implementação. O sexto, de analisar os benefícios em termos de organização e manutenibilidade, constitui o objeto central deste capítulo e foi conduzido, ainda que de forma qualitativa, nas seções anteriores. Desta maneira, considera-se que os objetivos específicos foram contemplados, em consonância com o objetivo geral de propor um manual de implementação da Clean Architecture em projetos Node.js com TypeScript.

5.6 LIMITAÇÕES E AMEAÇAS À VALIDADE

A interpretação dos resultados deve considerar um conjunto de limitações, que podem ser organizadas segundo as ameaças à validade usualmente reconhecidas em estudos dessa natureza. No tocante à validade externa, este trabalho constitui um estudo de caso único, centrado em uma aplicação de pequeno porte e de domínio simples, o que restringe a generalização das observações a outros contextos, escalas ou domínios, sobretudo a sistemas de maior complexidade, com múltiplos contextos de negócio e integrações externas. No tocante à validade de construto, os benefícios atribuídos à arquitetura foram avaliados de forma qualitativa e estrutural, sem a coleta de métricas quantitativas de manutenibilidade, como medidas de complexidade ou de acoplamento obtidas por ferramentas, nem a realização de experimentos controlados.

Acrescenta-se a ausência de uma linha de base para comparação, pois não foi implementada uma versão alternativa do sistema, sem a arquitetura, que permitisse contrastar diretamente os dois arranjos. Dessa forma, os ganhos discutidos derivam da análise dos princípios e de sua materialização no código, e não de um confronto empírico controlado. Ao final, cabe reconhecer que o desenvolvimento e a avaliação do sistema foram conduzidos pelo próprio autor, o que pode introduzir viés na apreciação dos resultados, configurando uma ameaça à validade interna. Tais limitações não invalidam as observações realizadas, mas delimitam seu alcance e apontam direções para trabalhos futuros, como a condução de estudos

comparativos, a adoção de métricas objetivas e a replicação da abordagem em sistemas de maior porte e por equipes independentes.

5.7 SÍNTESE DOS RESULTADOS

A título de síntese, o Quadro 1 relaciona os principais princípios da Clean Architecture às formas como foram concretizados no projeto e às evidências correspondentes.

Quadro 1 – Síntese da concretização dos princípios da Clean Architecture

Princípio	Concretização no projeto	Evidência
Regra de dependência	Dependências apontam das camadas externas para as internas; o núcleo desconhece a infraestrutura	Ausência de importações de framework no domínio e na aplicação (Seção 5.1)
Entidades com regras de negócio	Validações e comportamentos encapsulados nas entidades e no objeto de valor de identidade	Seção 4.1 e Figuras 7 a 20
Inversão de dependência	Casos de uso dependem de interfaces; implementações concretas residem na infraestrutura	Seções 4.3 e 4.4; substituíbilidade do repositório (Seção 5.1)
Separação de preocupações de borda	Validação, identificação de usuário e exceções tratadas na camada de adaptadores	Seção 4.3 e Figuras 34 a 41
Testabilidade	Caso de uso exercitado com repositório em memória, sem banco de dados	Seção 4.6 e Figuras 53 e 54

Fonte: elaborado pelos autores (2026).

De modo geral, os resultados sugerem que a adoção da Clean Architecture, embora implique maior esforço inicial de organização e planejamento, sobretudo em aplicações de pequeno porte, proporcionou um núcleo independente de tecnologias externas, substituível em seus detalhes técnicos e favorável à verificação automatizada. Esses ganhos, contudo, devem ser ponderados frente aos custos observados, como o maior número de componentes e a maior cerimônia para implementar funcionalidades simples. É plausível que a relação entre custo e benefício se torne mais favorável à medida que a complexidade e a longevidade do sistema aumentam, uma vez que é nesse cenário que os efeitos do acoplamento tendem a se manifestar com mais intensidade. Essa hipótese, todavia, demandaria verificação empírica em estudos de maior escala, o que reforça as direções de trabalho futuro apontadas na seção anterior.

6 CONCLUSÃO

Este trabalho teve como objetivo propor um manual de implementação da Clean Architecture em projetos Node.js com TypeScript, demonstrado por meio do desenvolvimento de uma API RESTful de gerenciamento de listas de tarefas. A motivação central residiu em discutir, a partir de uma implementação prática, as implicações da aplicação de princípios arquiteturais bem definidos para a organização, a manutenibilidade e a evolução de sistemas de software, ainda que aplicados a um domínio relativamente simples.

A partir do estudo da arquitetura de software e da Clean Architecture, procurou-se compreender a relevância da separação de responsabilidades, do controle de dependências e da proteção das regras de negócio diante de detalhes externos. Esses conceitos foram aplicados na construção da API, estruturada em camadas bem definidas, contemplando entidades, casos de uso, interfaces, adaptadores de apresentação e infraestrutura, conforme detalhado no manual que constitui o cerne deste trabalho.

A análise conduzida indica que a adoção da arquitetura favoreceu a construção de um sistema organizado e desacoplado. O domínio e os casos de uso mantiveram-se independentes de frameworks, de mecanismos de persistência e de protocolos de comunicação, ao passo que detalhes como o NestJS, o Prisma e o tratamento das requisições permaneceram restritos às camadas externas. Os resultados que sustentam essa observação foram discutidos no Capítulo 5, com destaque para a concentração das dependências de tecnologia na camada de infraestrutura.

A principal contribuição deste trabalho reside na sistematização de um material prático e didático sobre a aplicação da Clean Architecture em um projeto backend moderno. Ao associar conceitos teóricos a trechos reais de código, comentados camada a camada, o manual oferece um exemplo concreto de como estruturar uma aplicação e de qual é o papel de cada componente. Nesse sentido, o material pode servir como referência inicial para estudantes e desenvolvedores interessados não apenas na teoria, mas também na implementação da abordagem.

Cabe ainda ressaltar que a adoção de uma arquitetura bem definida não depende necessariamente da complexidade do domínio. Mesmo em uma aplicação simples, foi possível observar indícios de benefícios como maior clareza estrutural e melhor separação entre as regras de negócio e os detalhes técnicos. Tais observações sugerem que a Clean Architecture pode ser considerada desde as fases iniciais de um projeto, e não apenas em sistemas de grande porte.

A interpretação desses resultados deve considerar as limitações discutidas na Seção 5.6, entre as quais se destacam o caráter de estudo de caso único, o domínio reduzido, a ausência de medições quantitativas e a inexistência de uma linha de base para comparação. Soma-se a elas o fato de que o sistema não implementa um mecanismo completo de autenticação, valendo-se de um cabeçalho de identificação do usuário como forma simplificada, uma vez que o foco recaiu sobre a arquitetura da aplicação.

Como desdobramentos futuros, sugere-se a ampliação da aplicação com novas funcionalidades, como autenticação e autorização completas, compartilhamento de listas e filtros mais elaborados. Tais evoluções permitiriam avaliar a arquitetura em um cenário mais próximo de um ambiente de produção, com maior diversidade de regras de negócio e de integrações.

No tocante à verificação, a Seção 4.6 apresentou um teste de unidade ilustrativo de um caso de uso, conduzido sem dependência de banco de dados. A sua expansão para uma suíte abrangente, que contemple testes de unidade para as demais entidades e casos de uso, bem como testes de integração para os repositórios e adaptadores, permitiria evidenciar de maneira mais sistemática a testabilidade proporcionada pela arquitetura. A condução de estudos comparativos e a adoção de métricas objetivas, igualmente apontadas entre as limitações, constituem outra direção promissora.

Por fim, considera-se que o trabalho atingiu seu objetivo ao demonstrar, por meio de uma implementação prática, como os conceitos da Clean Architecture podem ser aplicados na construção de uma aplicação backend em Node.js e TypeScript. Ainda que o sistema desenvolvido tenha escopo reduzido, ele reúne os fundamentos necessários para a adoção da abordagem em projetos mais complexos e reforça a relevância da arquitetura de software na construção de sistemas sustentáveis.

REFERÊNCIAS

- ANICHE, M.; YODER, J. W.; KON, F.** Current challenges in practical object-oriented software design. In: INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING: NEW IDEAS AND EMERGING RESULTS (ICSE-NIER), 41., 2019, Montreal. Proceedings [...]. Piscataway: IEEE, 2019. p. 113-116.
- BAABAD, A.; ZULZALIL, H.; HASSAN, S.; BAHAROM, S.** Software architecture degradation in open source software: a systematic literature review. IEEE Access, v. 8, p. 173681-173709, 2020.
- BASS, L.; CLEMENTS, P.; KAZMAN, R.** Software architecture in practice. 4. ed. Boston: Addison-Wesley, 2021.
- BELTRÃO, A. C.; FARZAT, F. A.; TRAVASSOS, G. H.** Technical debt: a clean architecture implementation. In: CONGRESSO BRASILEIRO DE SOFTWARE: TEORIA E PRÁTICA (CBSOFT), 11., 2020, Natal. Anais estendidos [...]. Porto Alegre: Sociedade Brasileira de Computação, 2020.
- BIERMAN, G.; ABADI, M.; TORGENSEN, M.** Understanding TypeScript. In: EUROPEAN CONFERENCE ON OBJECT-ORIENTED PROGRAMMING (ECOOP), 28., 2014, Uppsala. Proceedings [...]. Berlin: Springer, 2014. p. 257-281.
- BROWN, P. S. et al.** Refactoring the Whitby intelligent tutoring system for clean architecture. Theory and Practice of Logic Programming, v. 21, n. 6, p. 818-834, 2021.
- CRESWELL, J. W.; CRESWELL, J. D.** Research design: qualitative, quantitative, and mixed methods approaches. 5. ed. Thousand Oaks: SAGE Publications, 2018.
- CUNNINGHAM, W.** The WyCash portfolio management system. ACM SIGPLAN OOPS Messenger, v. 4, n. 2, p. 29-30, 1992.
- EVANS, E.** Domain-driven design: tackling complexity in the heart of software. Boston: Addison-Wesley, 2003.
- FIELDING, R. T.** Architectural styles and the design of network-based software architectures. 2000. Tese (Doutorado em Ciência da Computação), University of California, Irvine, 2000.
- FOWLER, M.** Patterns of enterprise application architecture. Boston: Addison-Wesley, 2002.
- GIL, A. C.** Como elaborar projetos de pesquisa. 7. ed. Barueri: Atlas, 2022.
- GREBENUK, O.; PAVLENKO, V. S.** Architecture of ports and adapters in iterative development with limitations on time. Visnyk Universytetu Ukraina, 2019.
- HEVNER, A. R.; MARCH, S. T.; PARK, J.; RAM, S.** Design science in information systems research. MIS Quarterly, v. 28, n. 1, p. 75-105, 2004.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. ISO/IEC 25010: systems and software engineering, Systems and software Quality Requirements and Evaluation (SQuaRE), product quality model. Geneva: ISO, 2023.

KACZMARSKI, P. Web communication: implementing a RESTful web API in C# .NET 8 using clean architecture. *Communication & Cognition*, v. 57, n. 1-2, p. 3-42, 2024.

KRUCHTEN, P.; NORD, R. L.; OZKAYA, I. Technical debt: from metaphor to theory and practice. *IEEE Software*, v. 29, n. 6, p. 18-21, 2012.

LANO, K.; YASSIPOUR TEHRANI, S. Introduction to clean architecture concepts. Cham: Springer, 2023. p. 35-49.

MARTIN, R. C. Clean architecture: a craftsman's guide to software structure and design. Boston: Prentice Hall, 2017.

NUGROHO, Y. N.; KUSUMO, D. S.; ALIBASA, M. J. Clean architecture implementation impacts on maintainability aspect for backend system code base. In: INTERNATIONAL CONFERENCE ON INFORMATION AND COMMUNICATION TECHNOLOGY (ICOICT), 10., 2022, Bandung. Proceedings [...]. Piscataway: IEEE, 2022. p. 134-139.

PERRY, D. E.; WOLF, A. L. Foundations for the study of software architecture. *ACM SIGSOFT Software Engineering Notes*, v. 17, n. 4, p. 40-52, 1992.

PRESSMAN, R. S.; MAXIM, B. R. Engenharia de software: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021.

STEVENS, W. P.; MYERS, G. J.; CONSTANTINE, L. L. Structured design. *IBM Systems Journal*, v. 13, n. 2, p. 115-139, 1974.

WIERINGA, R. J. Design science methodology for information systems and software engineering. Berlin: Springer, 2014.